Combinatorial Methods in Algorithms

Lecture Summaries

Instructor: Dr. Or Zamir
Course Code: CS 0368-4246 (Spring 2026)

Scribes:
Or Sterb (Lecture 1)
Idan Izmirli (Lecture 2)
Amitai Hazan (Lecture 3)
Naomi Green (Lecture 4)
Yahel Caspi (Lecture 5)
Gabriel Ginzburg (Lecture 6)
Ron Vulkan (Lecture 7)
Nicolas Duek (Lecture 8)
Itai Halperin (Lecture 9)
Eitan Elbaum (Lecture 10)

Contents

Chapter 1

Pattern Detection and Extremal Graph Theory

1.1 introduction

In today's lecture we will introduce the problem of pattern detection and we will discuss how density of graphs affects it by discussing extremal graph theory.

1.2 Pattern Detection

Problem 1.1. *Given a large graph $G = (V, E)$ and a graph H of constant size, find whether G contains a copy of H.*

We wish to answer the following questions:

- In what complexity can the problem be solved as a function of $n = |V|$? As a function of $m = |E|$
- For what number of edges does H have to appear in the graph?
- Can we create a sparse graph that maintains whether G contains H or not?

First we need to decide how we represent a graph.

- Adjacency Matrix:
  - strengths: $O(1)$ to check whether a certain edge appears in the graph.
  - Weaknesses: $\Omega(n)$ to iterate over edges of a single vertex, $\Omega(n^2)$ space.
- Adjacency List
  - strengths: $O(d)$ to iterate over edges of a single vertex (where d is the degree of the vertex), $O(m + n)$ space.
  - Weaknesses: $\Omega(d)$ to check whether a certain edge appears in the graph.

We can improve it by instead of using linked list for each vertex we use a search tree of hash table. This brings the complexity of checking whether a certain edge is in the graph to $O(\log n)$ worst case, or $O(1)$ average case, respectively, which is what we will usually assume we have.

1.2.1 Triangle Detection

Note 1.2. *A triangle can be looked at as both a 3-cycle (C_3) or a clique of size 3 (K_3).*

Naive algorithm for triangle detection is to simply iterate over all triplets of vertices in the graph which has complexity $O(n^3)$.

A better approach can give $O(n^\omega)$ where it is the complexity of matrix multiplication and we know $2 \leq \omega \leq 2.372$. We denote by A the adjacenct matrix of G.

Algorithm 1.3. *Compute A^3 and the compute $\sum_{v \in V}(A^3)_{v,v}$. Return that there exists a triangle iff the sum is greater that 0. Note that the paths computed by powers of A are not simple but any path of size 3 from a vertex to itself has to be.*

Algorithm 1.4. *Compute A^2, then check where there exists a pair of vertices $(u, v) \in V \times V$ such that $(A^2)_{u,v} > 0$ (so there is a vertex $w \in V$ such that the path $u - w - v$ is in the graph) and also $(A)_{u,v} > 0$. If there exists such a pair, then there is a triangle in the graph ($u - v - w$).*

Similar algoritm to the second one check whether a 4-cycle is in the graph by checking whether there is a pair (u, v) such that $(A^2)_{u,v} > 1$.

1.2.2 Triangle Detection in Sparse Graphs

We now will assume $m << n^2$. There exists a naive algorithm in $O(mn)$, check for each pair $(v, e) \in V \times E$ if there exists edges from v to both vertices of e and if for some pair the answer is yes then there exists a triangle in the graph.

Theorem 1.5. *There exists a triangle detection algorithm in $O(m^{1.5})$.*

Proof. Let $d = m^{0.5}$. Any triangle which contains a vertex with degree larger than d we can find by the naive algorithm: check for every pair $(v, e) \in V_{deg>d} \times E$. The sum of all degrees in the graph is $2m$ (handshake theorem), so $|V_{deg>d}| * d \leq 2m$ and thus $|V_{deg>d}| \cdot m \leq 2m^2/d = O(m^{1.5})$. Now we want to find the triangle in which all vertices are of degree smaller than d. For each such edge (u, v) such that $deg(v) \leq d$ for each neighbour $w \in N(v)$, check if $(u, w) \in E$ and if it is there is a triangle. This part take $O(md) = O(m^{1.5})$, because for each edge we check at most d vertices. □

1.3 Extremal Graph Theory

Definition 1.6. *The extremal number (Turan Number) of a graph H, denoted $ex(H, n)$, is the maximal number of edges that a graph with n vertices can have without containing a copy of H. For every graph H, $0 \leq ex(H, n) \leq \binom{n}{2}$.*

What is the value of $ex(K_3, n)$? Note that in a bipartite graph there are no triangles (or any odd cycles), so $ex(K_3, n) \geq \lceil n/2 \rceil \cdot \lfloor n/2 \rfloor = \lfloor n^2/4 \rfloor$

Theorem 1.7. *$ex(K_3, n) = \lfloor n^2/4 \rfloor$ [15]*

Proof. We prove $ex(K_3, n) \leq \lfloor n^2/4 \rfloor$, which we do by induction. $ex(K_3, 2) = 1$, $ex(K_3, 3) = 2$. Now assume $ex(K_3, k) \leq \lfloor \lfloor k^2/4 \rfloor$ for all $k < n$. Let G be a graph of size n which does not contain a triangle. If there are no edges in G we are done. Otherwise let (u, v) be an edge in G. Note that $N(u) \cap N(v) = \emptyset$, because otherwise for $w \in N(u) \cap N(v)$ $u - v - w$ is a triangle. This means $deg(u) + deg(v) \leq nj$. If we delete both u and v and all edges containing them we are left with a graph containing at most $\lfloor (n-2)^2/4 \rfloor$ due to the induction hypothesis. We deleted at most $n - 1$ edges as the edge (u, v) is counted twice in $deg(u) + deg(v)$ and so G has at most $\lfloor (n-2)^2/4 \rfloor + n - 1 = \lfloor n^2/4 \rfloor$. □

An alternative proof is as follows:

Proof. $mn \geq \sum_{(u,v)\in E}(deg(u) + deg(v)) = \sum_{v\in V} deg(v)^2 = n(\frac{1}{n}\sum_{v\in V} deg(v)^2$ (using Jensen's inequality) $\geq n(\frac{1}{n}\sum_{v\in V} deg(v))^2 = n(\frac{2m}{n})^2 = \frac{4m^2}{n}$. From here we can get $m \leq \frac{n^2}{4}$. □

Theorem 1.8. *for every $r \geq 2$, $ex(K_{r+1}, n) = 0.5(1 - \frac{1}{r})n^2$ [22]*

We show lower bound:

Proof. Let G be a graph which consists of r sets of vertices, each of size n/r. A pair of vertices $(u, v) \in V \times V$ is an edge iff u and v are in different sets. The number of edges in this graph is $\binom{r}{2}(\frac{n}{2})^2 = 0.5(1 - \frac{1}{r})n^2$. There is no K_{r+1} in this graph, as if there was, then at least two vertices are in the same set according to the pigeon-hole principle, but then there is no edge between those vertices. Note: if r does not divide n then the graph is created with sets of size $\lfloor n/r \rfloor$ and $\lceil n/r \rceil$. □

Definition 1.9. *The chromatic number of a graph H, denoted $\chi(H)$, is the minimal number of colours we can use to colour the vertices of H such that no edge is monochromatic.*

Theorem 1.10. *$ex(H, n) = 0.5(1 - \frac{1}{\chi(H)-1})n^2 + o(n^2)$. [7]*

Conclusion 1.11. *All graph which hold $ex(H, n) = o(n^2)$ are bipartite. For example: even cycles, trees, $K_{a,b}$.*

Theorem 1.12. *Let P_k denote paths of size k, $ex(P_k, n) \leq kn$*

Lemma 1.13. *Let G be a graph with more than kn edges, then there is a subgraph of G in which all degrees are at least k.*

Proof. Let's define an algorithm in which if there is a vertex in a graph of degree below k, deletes the vertex. The algorithm only stops when no such vertex exists. Notice that at the beginning the average degree in G is $\frac{2n}{m} \geq 2k$. Assume before deleting a vertex there were n' vertices in the graph, then the sum of degrees is at least $2kn'$. After the deletion the sum of the degrees is at least $2kn' - 2k = 2k(n' - 1)$, and the average degree is at least $2k$. The algorithm thus cannot end with a graph of size less than $2k$, and particularly the graph at the end is not empty. □

Now we prove the theorem using the lemma:

Proof. Let's take a subgraph of G where all the vertices are at least k. Now simply build the path greedily: start from any vertex. From each vertex go to a vertex that has not been visited yet. Before the path is size k, there must be a neighbour of the current vertex which has not yet been included in the path (because of the degree requirements). □

Note 1.14. *$ex(P_k, n) = \Omega(kn)$. Take a graph with $\frac{n}{k-1}$ cliques with size $k-1$.*

Theorem 1.15. *$ex(C_4, n) = \Theta(n^{1.5})$*

Proof. **Upper Bound**: The main idea is the fact that there exists a C_4 iff there are two P_2's with the same ends. Thus, if the number of P_2's is $\binom{n}{2} + 1$, there exists a C_4 in the graph. $\#P_2 = \sum_{v \in V} \binom{deg(v)}{2} = n(\frac{1}{n}\sum_{v \in V} \binom{deg(v)}{2}) \geq$ (using Jensen's inequality) $n\binom{\frac{1}{n}\sum_{v \in V} deg(v)}{2} = n\binom{\frac{2m}{n}}{2} \geq \frac{n}{2}\frac{2m}{n}(\frac{2m}{n} - 1) = \frac{2m^2}{n} - m$. So if $m = \omega(n^{1.5})$ then $\#P_2 > \binom{n}{2}$, so there is a C_4 in the graph. **Lower Bound**: We creat a bipartite graph $G = (V = A \cup B, E)$, such that $|E| = \Omega(|V|^{1.5}$, but the graph contains no C_4's. Take a prime $p \approx \sqrt{n}$. A will consist of p^2 vertices representing the points in $\mathbb{Z}_p^2$. B will represent the set of all line in $\mathbb{Z}_p^2$, which each representing a line $L = ax + b$ where $(a, b) \in \mathbb{Z}_p^2$, and will have an edge to all vertices in A that are of the type $(x, ax + b)$. There are $p^2 \approx n$ possible lines (every combination of a and b). As for edges, each vertex in B is connected to exactly p nodes in A, which means the overall number of edges is $p^3 \approx n^{1.5}$. The graph has $2p^2$ vertices and p^3 edges so $E = \Omega(|V|^{1.5})$. For any two different lines $L_1 = a_1x + b_1, L_2 = a_2x + b_2 \in B$, and see in how many vertices they intersect. If $ax_1 + b_1 = a_2x + b_2$ if $a_1 \neq a_2$ we have $x = -\frac{b_2 - b_1}{a_2 - a_1}$ and the lines intersect at a single point. if $a_1 = a_2$ then $b_1 = b_2$ so these are the same vertices. Thus in the graph no two distinct vertices in B share more than one different neighbout in A, so there are no C_4's in the graph. □

Theorem 1.16. *There is an $O(n^2)$ algorithm for C_4 detection.*

Proof. We create a hash table which contains a mapping from pairs $(u, v) \in V \times V$ to vertices $w \in V$ for which the path $u - w - v$ is in the graph. At the beginning the table is empty. The algorithm goes to each vertex w, and for each pair (u, v) neigbours of w. we insert w to the hash table with the key (u, v). We stop the algorithm the first time that before insertion of w_2, the list for a certain pair (u, v) is not empty (say it contains w_1) and we return that there is a C_4: $u - w_1 - v - w_2$. If we passed all the nodes and this did not happen, return that there is no C_4. Notice that each pair is accessed at most twice, so the algorithm takes $O(n^2)$. □

Theorem 1.17. *$ex(C_{2k}, n) \leq 10n^{1+\frac{1}{k}}$ [4]*

The proof will be shown later in the course. **Open Question**: Is this a tight bound? Only knows for C_4, C_6, C_{10}

Theorem 1.18. *if $m \geq 10n^{1+\frac{1}{k}}$, then girth$(G) \leq 2k$ (this means there is a cycle of size at most $2k$).*

Proof. From Lemma 1.12 we can take a subgraph with a minimal degree of $10n^{\frac{1}{k}}$. Take any vertex and start a BFS from it. In the first level we have at least $10n^{\frac{1}{k}}$ vertices. Afterwards, if any two vertices in the first level are connected there is a C_3. Otherwise, the second level is

of size at least $(10n^{\frac{1}{k}}-1)^2$ as if two vertices in the first level have a mutual neighbour there is a C_4. We continue this for k levels, each time if there is an edge between two vertices in the same level there is a cycle of size less than $2k$ and for same reason no two vertices in the same level can have a mutual neighbour in the next level. Thus the i-th level has at least $(10n^{\frac{1}{k}}-1)^i$ so level k has too many vertices, thus at some point there was a repetition of vertices, which proves there is a cycle of size at most $2k$ in the graph. □

1.4 Spanners

Definition 1.19. *For a graph G, a spanner is a subgraph $G' \subseteq G$ which approximates shortest path sizes for G, and G' is sparse. For every $u, v \in V$, $dist_G(u,v) \leq dist_{G'}(u,v) \leq t \cdot dist_G(u,v)$, and t is the "stretch factor" of the spanner.*

Theorem 1.20. *Every graph G has a spanner with $|E| = O(n^{1+\frac{1}{k}})$ and $t = 2k-1$.*

Proof. We build G' algorithmically: start from an empty set for G'. We transverse the edges in E one by one and add an edge (u,v) to $E(G')$ iff currently $dist_{G'}(u,v) \geq 2k$. First we show $t = 2k-1$. For any path in G, every edge in th epath is either in G' or has an alternative path in G' of size at most $2k-1$ (if there isn't such a path there wouldn't have been when we reached the edge in the algorithm and we would have added it to G'). Now we show $|E| = O(n^{1+\frac{1}{k}})$. Notice the graph has no cycles of length $\leq 2k$ as if it had, the last edge added shouldn't have been added according to the algorithm as it has an alternate path of length $< 2k$.

□

Chapter 2

Randomized Algorithms for Pattern Finding

Date: April 20, 2026
Scribe: Idan Izmirli

2.1 Reminder and Overview

Last time we talked about algorithms for finding simple subgraphs. We learned how to find C_3 in $O(n^\omega)$ time, and saw in an exercise that it can also be done in $O(m^{\frac{2\omega}{1+\omega}})$ time, which can be better for sparse enough graphs. We also saw how to find C_4 in $O(n^2)$ time, but did not find a better algorithm for sparse graphs. We then talked about extremal graph theory, and even saw an example of how to use it in an algorithm, by using a spanner.

Today we will look at algorithms that use randomness more heavily than before.

Notation: Throughout this lecture, P_k denotes a simple path on k vertices (and hence on $k-1$ edges), and C_k denotes a simple cycle on k vertices (and hence on k edges).

2.2 How to Find a Square

Claim 2.1. *In a graph where $m \geq 2n^{\frac{3}{2}}$, we can find a C_4 in expected time $O(m)$.*

Proof. We start by proving the following lemma.

Lemma 2.2. *Given a graph G and an edge e, there is a deterministic $O(m)$ algorithm that finds a C_4 containing e, if one exists.*

Proof. Write $e = (u, v)$ and look at all other edges $e' = (w, x)$. If (u, v, w, x) or (u, v, x, w) is a C_4, we return it. If after looking at all possible e' we have not found a C_4, then there is no C_4 that contains e. □

Using this lemma, a simple algorithm comes to mind: choose a random edge e until we find one that is contained in a C_4. In order for this algorithm to be efficient, we need to prove that many edges are part of a C_4.

Lemma 2.3. *If $m \geq 2n^{\frac{3}{2}}$, then there are at most $\frac{m}{2}$ edges that are not part of a C_4.*

Proof. Let E' be the set of all edges that are not part of a C_4. The subgraph containing only the edges of E' does not contain any C_4, and therefore $|E'| \leq n^{\frac{3}{2}} \leq \frac{m}{2}$. □

We now know that if we choose a random edge, the probability that it is contained in some C_4 is at least $\frac{1}{2}$. That means that if we run the simple algorithm we suggested, choosing a random edge until we find one that is contained in a C_4, the number of iterations is a geometric random variable with success probability at least $\frac{1}{2}$. In particular, the expected number of iterations is at most 2, and therefore we are done. □

Observation 2.4. *This proof is not very specific to C_4. The first lemma can be modified to work for other graphs, and in the second lemma only the extremal number needs to be changed to the matching one.*

And what about sparse graphs?

Claim 2.5. *There is an algorithm that finds a C_4 (if one exists) in $O(m^{\frac{4}{3}})$ expected time.*

Before we prove this, note that for $m > n^{\frac{3}{2}}$ we know that a C_4 always exists, and that $m = n^{\frac{3}{2}}$ is also the exact point at which this algorithm has the same complexity as our old $O(n^2)$ algorithm, so this complexity is quite natural.

Proof. We will use the following algorithm: while there is a node of degree $< 10m^{\frac{1}{3}}$, we remove it.

Note that this can be done in time $O(m)$ by maintaining a list of nodes with degree lower than $10m^{\frac{1}{3}}$, where each time we remove a node we look at all of its neighbors and check whether they should enter the list.

By the end of this process we are left either with a graph of minimum degree at least $10m^{\frac{1}{3}}$, or with an empty graph.

In the first case, let m' be the new number of edges and n' the new number of nodes. Since every remaining node has degree at least $10m^{\frac{1}{3}}$, and since $m' \leq m$, it holds that

$$m' \geq \frac{1}{2}n' \cdot 10m^{\frac{1}{3}} = 5n'm^{\frac{1}{3}} \geq 5n'(m')^{\frac{1}{3}}$$

and therefore

$$(m')^{\frac{2}{3}} \geq 5n' \implies m' \geq (5n')^{\frac{3}{2}} \geq 2(n')^{\frac{3}{2}},$$

so we can apply the previous algorithm to the remaining graph.

In the second case, we are left with an empty graph. This means that during the process of removing nodes of degree less than $10m^{\frac{1}{3}}$, all edges were removed. We can now turn our graph into a directed graph, where each edge is directed out of the node whose removal deleted it. In this directed graph, the out-degree of each node is at most $10m^{\frac{1}{3}}$. We call this a graph of "low degeneracy".

In our new directed graph we go over all edges $e = (u, v)$. For each of them we look at all edges that go out of u towards some vertex w. This gives us a path on three vertices (v, u, w) (not all edges are necessarily directed consistently, but this is not important to us). We similarly look at all edges that go out of v, forming paths of the form (w, v, u). We save all of these P_3's in a hash table, and stop when we have found two different P_3's between the same two nodes, which form a C_4.

The time complexity of the algorithm is $m \cdot O(m^{\frac{1}{3}}) = O(m^{\frac{4}{3}})$, so all that remains is to prove that it works.

When forming P_3's, our algorithm goes over all P_3's in which at least one of the edges points from the middle node outwards. In other words, we go through all possible (u, v, w) that are not of the form $(u \to v \leftarrow w)$. All we need to show is that if there is a C_4, then it can be broken into two P_3's that are not of that form.

Let (a, b, c, d) be a C_4. If $(a, b, c), (c, d, a)$ is not a good partition, it means that either $(a \to b \leftarrow c)$ or $(a \to d \leftarrow c)$. In either case we know that there is at least one edge of the C_4 that goes out of a, and at least one that goes out of c, meaning that $(b, a, d), (d, c, b)$ is a good partition.

In both cases we were able to find a C_4 (if one exists) in $O(m^{\frac{4}{3}})$ time, and therefore we are done. □

2.3 DAGs

In the past we tried to create algorithms for longer paths and cycles, and encountered the problem of paths that are not simple. For instance, the $O(n^\omega)$ algorithm could count the number of paths of a specific length when allowing repeated vertices, but not the number of simple paths. We would like to fix this, but first we need to present the following trick, which we will use from now on.

Claim 2.6. *Given an algorithm with expected running time T that always answers "no" when the answer is "no", and answers "yes" with probability at least p when the answer is "yes", we can devise an algorithm with $O\left(\frac{xT}{p}\right)$ expected running time that always answers "no" when the answer is "no", and answers "yes" with probability at least $1 - e^{-x}$ when the answer is "yes".*

Proof. We simply repeat the algorithm $\frac{x}{p}$ times, and answer "yes" if any of the iterations answered "yes". This algorithm always returns "no" on a "no" instance, and on a "yes" instance the probability of returning "yes" is at least

$$1 - (1 - p)^{\frac{x}{p}} \approx 1 - e^{-x}.$$

□

Note: This also works for returning a specific instance of a subgraph if one exists, not only for answering "yes" or "no".

Armed with this trick, we would like to solve our "simple paths only" problem. First, we notice that many of these problems are easier to solve on a DAG (directed acyclic graph). For instance, all paths are simple, and there are no cycles at all. There are also useful computations we can perform on a DAG.

Claim 2.7. *Given a DAG G and a specific node u, it is possible in time $O(m)$ to compute, for all nodes v, the length of the longest path from u to v, as well as to find an instance of such a longest path.*

Proof. We start by performing a topological sort on G, so that we may now refer to the nodes of the graph as $v_1, v_2, \ldots, v_n$ such that all edges go from the node with the smaller index to the one with the larger index. Write $u = v_x$.

Now, for all i we would like to compute a path $d[i]$ that is a longest path from u to v_i, if such a path exists.

For $i < x$ such a path does not exist. For $i = x$ the longest path is the empty path from u to itself. For $i > x$, we look at all j such that there is an edge from v_j to v_i, and consider the path

$$d[j] + (v_j, v_i)$$

(if $d[j]$ exists). The longest path among all of these is a valid value for $d[i]$. This means we can use dynamic programming to compute $d[i]$ for all i in order. This takes $O(m)$ time, since the possible values of j for each i correspond to the edges that end at v_i. □

Claim 2.8. *It is possible to find a P_k in a DAG in time $O(m)$ (if one exists).*

Proof. We add a fictitious node with an edge from it to every node. We then find a longest path from it to any node. A path on at least $k + 1$ vertices contains a P_k in the original DAG.

Also, any P_k in the original DAG can be turned into a path on $k + 1$ vertices by adding the new node at the start, so if there is a P_k, we are indeed guaranteed to find one. □

Claim 2.9. *Given a DAG G, it is possible in time $O(\log(k) \cdot n^\omega)$ to determine, for all u, v, whether there is a path on exactly k vertices from u to v.*

Proof. We raise the adjacency matrix to the $(k-1)$-st power, which by repeated squaring takes only $O(\log k)$ matrix multiplications. Entry (u, v) of the result is then the number of paths on exactly k vertices from u to v (in a DAG every walk is a simple path). □

Claim 2.10. *Given a directed graph G and a node u, it is possible in time $O(mk)$ to determine, for each node v and each $i \leq k$, whether there is a path on exactly i vertices from u to v.*

Proof. For a node v and a value $1 \leq i \leq k$, define $d[v][i]$ to be true if and only if there is a path on exactly i vertices from u to v. For $i \geq 2$ we notice that $d[v][i]$ is true if and only if there is an edge from some w to v such that $d[w][i-1]$ is true. The only v for which $d[v][1]$ holds is $v = u$. This allows us to easily compute $d[v][i]$ for all v, i in increasing order of i. □

So we are convinced that DAGs are useful; now let us go back to regular graphs and use what we have learned.

Note: From now on, all of our algorithms will be probabilistic, meaning that they have a non-zero error probability. Specifically, they will never return the subgraph we are looking for if no instance of it exists, and if an instance does exist, they will return one with probability $1 - \delta$ for an arbitrarily small (but constant) δ. We will usually take $\delta = e^{-100}$, but of course this can be amplified even further.

Claim 2.11. *It is possible to check whether a graph has a P_k in time $O(k!\, m)$.*

Proof. We randomize an order of the vertices, and direct every edge from the node with the smaller index towards the node with the larger index. This means we have turned our graph into a DAG, and we can use the previous algorithm to find a P_k in it, if one exists, and return it.

To analyze our success probability when a P_k exists, let us look at a specific P_k. If its k vertices appear in the correct order relative to each other, it becomes a path in the DAG, so our success probability is at least $\frac{1}{k!}$.

We then apply our trick from the beginning of this section and obtain an $O(k!\, m)$ algorithm with failure probability e^{-100}, and thus we are done. □

Claim 2.12. *Given a graph G and a node u, it is possible in time $O(k!\, m \log(n))$ to check, for every v, whether there is a P_k between u and v, with failure probability $o(1)$.*

Proof. We randomize an order of the vertices in which u is first, and check whether there is a P_k from u to v in time $O(m)$. The probability that we detect any specific P_k is at least $\frac{1}{k!}$. We run this $\log(n) \cdot 100 \cdot k!$ times, and for each v the probability of never finding a P_k between u and v when one exists is bounded by $e^{-100\log(n)} = n^{-100}$. Using the union bound, the probability that there exists any node v such that a P_k between u and v exists but was never found in any iteration is bounded by n^{-99}, and thus we are done. □

Claim 2.13. *It is possible to find a C_k in $\tilde{O}(k!\, mn)$ time.*

Proof. We start by performing the previous algorithm from every u, in order to determine whether there is a P_k between any pair u, v. Now we go over all edges $e = (u, v)$, and if there is a P_k between u and v, we close it using e and obtain a C_k. If we have not found any such pair, then there is no C_k. □

Claim 2.14. *It is possible to find a C_k in $\tilde{O}(k!\, n^\omega)$ time.*

Proof. We randomize an order of the nodes and turn the graph into a DAG. We check in time $O(\log(k) \cdot n^\omega)$ which nodes have a P_k between them. We repeat this $k! \cdot 100 \cdot \log(n)$ times. The probability of detecting a specific P_k in a single iteration is at least $\frac{1}{k!}$, so the probability of missing it in all iterations is at most n^{-100}. The probability that there exists a pair u, v with a P_k between them that we never detect is therefore bounded by n^{-98}. From here we know whether there is a P_k between any u, v, so we continue as in the previous algorithm. □

This is all well and good, but we notice that the dependence on k is still rather high. If we believe that finding a Hamiltonian path is difficult, we cannot expect to do better than exponential in k, but we would still like to reach a complexity of the form

$$e^{\Theta(k)} \cdot \mathrm{poly}(n, m, k),$$

and for that we will need a new method.

2.4 Color-Coding

The technique presented in this section is due to Alon, Yuster, and Zwick [2].

Instead of directing the edges, let us color the nodes, and require that the path does not pass through the same color twice.

Definition 2.15. *A path in which all the nodes have distinct colors is called a* rainbow path.

This is promising for two reasons:

- A rainbow path must be a simple path, since if a node appeared twice then its color would also appear twice.
- The probability that a given P_k becomes a rainbow path under a random coloring with k colors is relatively high. By Stirling's approximation it is

$$\frac{k!}{k^k} \approx \frac{\sqrt{2\pi k}\left(\frac{k}{e}\right)^k}{k^k} = \frac{\sqrt{2\pi k}}{e^k} \geq e^{-k}.$$

So all we really need to do is find efficient ways to detect rainbow paths.

Claim 2.16. *Given a colored graph G with k colors and a node u, it is possible in time $O(2^k m)$ to find the longest rainbow path from u to any node v.*

Proof. We once again use dynamic programming.

Denote the color of a node v by $c(v)$. Let C be the set of colors, and for any $A \subseteq C$ and node v, let $d[A][v]$ be a rainbow path from u to v that uses exactly the colors in A, if such a path exists. Now, for any A and v, we notice that a path for $d[A][v]$ can only exist if $c(v) \in A$, and in that case it must be a path of the form

$$d[A \setminus \{c(v)\}][w] + (w, v)$$

for some w that is a neighbor of v. Additionally, a path of this form is always valid as $d[A][v]$. We also notice that the only A with $|A| = 1$ for which some $d[A][v]$ exists corresponds to the empty path from u to itself, namely $d[\{c(u)\}][u]$.

Using this, we can apply dynamic programming to compute the values of $d[A][v]$ for all A and v, by going over all $A \subseteq C$ in increasing order of cardinality.

Finally, the longest rainbow path to a specific node v is the longest $d[A][v]$ we found. □

Observation 2.17. *By changing the initial condition to allow the empty path from any node v to itself as $d[\{c(v)\}][v]$, we obtain the longest rainbow path ending at each node, instead of forcing a specific starting node. This also allows us to find the longest rainbow path overall.*

Claim 2.18. *It is possible to find a P_k in $\tilde{O}((2e)^k m)$ time.*

Proof. We randomize a coloring with k colors and find the longest rainbow path in $O(2^k m)$ time. The probability that any specific P_k becomes a rainbow path is at least e^{-k}, and so we repeat this $\Theta(e^k)$ times to achieve failure probability e^{-100}, and thus we are done. □

Claim 2.19. *It is possible to find a C_k in $\tilde{O}((2e)^k mn)$ time.*

Proof. We randomize a coloring with k colors, and for every node u we find the longest rainbow paths from it to any node v in $O(2^k m)$ time. We then go over all edges $e = (u, v)$ and check whether the longest rainbow path between u and v spans k vertices. If so, we have found a C_k. If no edge could be completed into a C_k, then there are no rainbow cycles in this graph.

The probability of a cycle becoming a rainbow cycle is the same as that of a path becoming a rainbow path. This means that, as before, we can repeat this $\Theta(e^k)$ times, and be done. □

2.5 Matricization

We want to use the fact that matrix multiplication is fast, so we have to involve matrices somehow. Let us define some.

Let i be a color. Set R_i to be the matrix whose columns that match vertices with color i are all 1, and whose other columns are all 0. Set A_i to be the matrix whose entry at index (u, v) is 1 if there is an edge between u and v, and also u is of color i and v is not of color i; otherwise the entry is 0.

We notice that

$$A_1 A_2 \cdots A_{k-1} R_k$$

is a matrix whose entry at index (u, v) is the number of paths from u to v that pass exactly through the colors $1, 2, \ldots, k$ in that order.

Using matrices of this kind, we can determine whether there is a rainbow path between any two nodes using the matrix

$$\sum_{\pi \in S_n} A_{\pi(1)} A_{\pi(2)} \cdots A_{\pi(k-1)} R_{\pi(k)}$$

whose entry at index (u, v) is the number of rainbow paths on k vertices between u and v. Now we only need to compute it.

Claim 2.20. *We can compute this matrix in $\tilde{O}(k2^k n^\omega)$ time.*

Proof. Define $S(A)$ to be the set of bijections from $[|A|]$ to A. Let C be the set of all colors. For $B \subseteq C$, define

$$f(B) = \sum_{\pi \in S(B)} A_{\pi(1)} A_{\pi(2)} \cdots A_{\pi(|B|)}.$$

We are interested in computing the sum

$$\sum_{c \in C} f(C \setminus \{c\}) R_c,$$

so it is enough to show how to compute all values of f.

We notice that for all $B \subseteq C$,

$$f(B) = \sum_{b \in B} f(B \setminus \{b\}) A_b,$$

and for all $c \in C$,

$$f(\{c\}) = A_c.$$

Thus we can use dynamic programming to compute all $f(B)$ in increasing order of cardinality. This takes $\tilde{O}(k2^k n^\omega)$ time. □

As an immediate consequence we get the following.

Claim 2.21. *We can find a C_k in $\tilde{O}(k(2e)^k n^\omega)$ time.*

Proof. We randomize a coloring and use the previous algorithm to determine, in $\tilde{O}(k2^k n^\omega)$ time, whether there is a rainbow path on k vertices between any two nodes. We then check whether there is any edge $e = (u, v)$ such that there is a rainbow path on k vertices between u and v. We repeat this $\Theta(e^k)$ times, for failure probability at most e^{-100}. □

2.6 Summary

To summarize:

- We know how to find a C_3 in $O(n^\omega)$ or $O(m^{\frac{2\omega}{1+\omega}})$ time.
- We know how to find a C_4 in $O(n^2)$ or $O(m^{\frac{4}{3}})$ time.
- We know how to find a P_k in $\tilde{O}((2e)^k m)$ time.
- We know how to find a C_k in $\tilde{O}((2e)^k mn)$ or $\tilde{O}(k(2e)^k n^\omega)$ time.

2.7 De-Randomization

Suppose that we want a deterministic algorithm.

Observation 2.22. *It would be enough to have a family F of colorings such that for every subset $U \subseteq V$ of size k there is a coloring $f \in F$ that gives all nodes in U distinct colors. This property of F does not depend on the graph at all.*

We saw that if we randomize $e^k \log(n^k) = ke^k \log(n)$ colorings, we obtain such a family with probability > 0, and therefore one always exists.

This observation gives us a non-uniform deterministic algorithm for finding a P_k and a C_k (meaning an algorithm that depends on n and k in a way that is not necessarily computable).

2.8 Θ-Graphs

Definition 2.23. *A graph comprised of two cycles that share a single edge is called a* Θ-graph.

The girth of a Θ-graph is, as in any graph, the minimum length of a cycle. In the case of a Θ-graph, it is the length of the shorter of the two cycles that comprise it.

Claim 2.24. *If G is a graph with $m \geq 2tn$, then G has a subgraph that is a Θ-graph with girth $\geq t + 1$, and it can be found in $O(m)$ time.*

Proof. We start by finding a subgraph of minimum degree $\geq 2t$, which we already know can be done efficiently.

In this subgraph, let us find some maximal path, meaning a path $(v_1, v_2, \ldots, v_r)$ where no additional node can be appended after v_r. In other words, all of the neighbors of v_r already lie on the path, which implies in particular that $r \geq 2t + 1$. A path such as this can easily be found by repeatedly adding nodes to the end for as long as it is possible.

Let $i_1 < i_2 < \cdots < i_x$ be the indices of v_r's neighbors on the path. Note that $x \geq 2t$. Now let $j = i_1$ and $k = i_t$. It holds that $j + t - 1 \leq k$, as there are $t - 1$ neighbors of v_r between them. Similarly, $k + t \leq r$. Now we look at the following cycles:

$$(v_j, v_{j+1}, \ldots, v_k, v_r)$$

$$(v_k, v_{k+1}, \ldots, v_r)$$

They are both of length at least $t + 1$ and share a single edge (v_k, v_r), and thus they form a Θ-graph with girth at least $t + 1$. $\square$

Chapter 3

Even-Length Cycles and Lower Bounds for Algorithms

Date: May 4, 2026
Scribe: Amitai Hazan

3.1 Introduction

In the first part of this lecture, we further explore the properties of theta-graphs, and use these results to construct an $O_k(n^2)$ algorithm for finding even-length cycles and prove the Bondy-Simonovits theorem. In the second part, we introduce fine-grained complexity and present some basic examples concerning the SETH hypothesis.

3.2 Theta Graphs

3.2.1 Reminders

A *theta-graph* is a graph obtained from two cycles sharing a single edge. A theta-graph has a girth greater than t if both cycles that form it have length greater than t. In the previous lecture we proved the following theorem:

Theorem 3.1. *If a graph has $m \geq 2tn$, then it contains a theta-graph as a subgraph of girth greater than t, and we can find it in $O(m)$ time.*

3.2.2 Periodic Coloring

Definition 3.2. *Given a graph and a coloring of its vertices $c : V \to [k]$, we say that the coloring is* t-periodic *if for every P_t in the graph, its two endpoints have the same color.*

Claim 3.3. *For a cycle C_l and a coloring of its vertices, the smallest t^* such that the coloring is t^*-periodic divides the cycle length l.*

Proof. Let t^* be the smallest period of the coloring. Since $t^* < l$ we can write $l = \lfloor \frac{l}{t^*} \rfloor \cdot t^* + r$ where $0 \leq r < t^*$. Assume by contradiction that $r > 0$. Consider two vertices on the cycle at

distance r. There are two simple paths between them: one of length r and the other of length $\lfloor \frac{l}{t^*} \rfloor \cdot t^*$. Since the coloring is t^*-periodic, the existence of the latter path implies that the two vertices are of the same color. Since this happens for every two vertices at distance r, we get that the graph is r-periodic, contradicting the minimality of t^*. Therefore, t^* must divide l. □

Claim 3.4. *Let H be a theta-graph with girth $> t$ and a t-periodic coloring. If one of its three cycles is t^*-periodic for some $t^* \leq t$, then all of its cycles are t^*-periodic.*

Proof. Denote the t^*-periodic cycle by C, and consider any two vertices u, v at distance t^* on one of the two other cycles. Since the girth is greater than t, we can find two vertices u', v' on C at distance t^* such that there are simple paths $u \rightsquigarrow u', v \rightsquigarrow v'$, both of length divisible by t. Since C is t^*-periodic, u' and v' are of the same color. Since H is t-periodic, u and u' are of the same color, and so are v and v'. It follows that u and v are of the same color as well. □

Claim 3.5. *If a theta-graph has girth $> t$ and a t-periodic coloring, then it is also 2-periodic.*

Proof. Denote the lengths of the two cycles forming the graph by a, b. Then the length of the third cycle is $a + b - 2$. Let t^* be the (joint) minimal period of the three cycles. t^* divides both a, b and $a+b-2$. Using modular arithmetic we conclude that t^* also divides $a+b-(a+b-2) = 2$, hence $t^* \leq 2$ and $t^* \in \mathbb{N}$. The only options for t^* are 2 or 1. Since every 1-periodic graph is also 2-periodic, we get that anyways the graph is 2-periodic. □

3.3 Finding C_{2k}

First, we shall use our knowledge of theta-graphs to prove the following lemma:

Lemma 3.6. *Let G be a connected graph with more than $2tn$ edges, whose vertices are colored with three colors. Then G contains a path P_t whose endpoints are colored differently. If G is not bipartite, two colors suffice. Moreover, such a path can be found in time $O(m)$.*

Proof. Since $m > 2tn$, the graph contains a theta-graph H with girth $> t$ (which can be found in $O(m)$ time). Note that we can check in $O(n)$ time whether H is t-periodic: there are at most eight simple paths of length t originating from each vertex. Hence, we perform the check by storing the vertices of H in a suitable array and iteratively traverse the $t+1$ consecutive entries corresponding to each such path. If H is not t-periodic, it means we found a P_t with differently colored endpoints and we are done. Otherwise, H is t-periodic. Hence, H is also 2-periodic, so it is colored with at most two colors and we can fix a vertex v in G colored differently. Using BFS, we find the shortest path P from v to H (recall that G is connected so such a path must exist). P is a simple path that touches H only at its endpoint. Since the girth of H is greater than t, we can extend P inside H to get a simple path P' whose length is a multiple of t. Note that the endpoints of P' are colored differently. By decomposing P' into a sequence of simple paths of length t, we are guaranteed that at least one of them has differently-colored endpoints.

We turn to the special case where G is not bipartite and colored with only two colors. Then G must have a monochromatic edge (u, v). As before, we consider the theta-graph H. If H is colored with a single color, we are done as in the previous case. Otherwise, find a shortest path P from u to H. $u, v \notin V_H$, because a 2-periodic graph that has a monochromatic edge is also 1-periodic, and we assumed that H is not single-colored. We may assume P does not pass through v, as at least one of u, v has a path to H that avoids the other. We prepend (u, v) to

P and extend it inside H to get a path P' from u to a vertex u' in H whose length is a multiple of t. By removing (u, v) and appending an edge (u', v') in H we get another path P'' of the same length from v to v'. Note that since H is 2-periodic (and has exactly two colors), u', v' are of different colors. It follows that either P' or P'' has differently-colored endpoints, and we can complete the proof as before. □

Theorem 3.7. *For any fixed k, it is possible to find a C_{2k} in a graph in time $O_k(n^2)$ [25].*

Proof. Given a vertex v, we describe an $O_k(n)$ time algorithm that either finds some C_{2k} in G or certifies that no C_{2k} passes through v. Running it from each vertex will give us the desired $O_k(n^2)$ time.

Run BFS "slowly" from v. Maintain the following values throughout the traversal:

- L_i - vertices of distance i from v we've so far discovered.
- $E(L_i)$ - the edges within each L_i.
- $E(L_i, L_{i+1})$ - the edges between L_i and L_{i+1}.

If any of the following conditions holds, immediately terminate the BFS:

(a) We have finished traversing G or finished traversing L_{k-1}.

(b) For some i, $4k|L_i| < E(L_i)$.

(c) For some i, $4k(|L_i| + |L_{i+1}|) < E(L_i, L_{i+1})$.

For each condition above, we describe the algorithm's next steps:

(a) We are guaranteed to have discovered all the edges of any C_{2k} that passes through v, because the farthest vertex in such a C_{2k} from v is of distance $k-1$ (and we have fully explored up to layer L_{k-1}). Moreover, since conditions (b),(c) did not occur, the total number of edges discovered is $O(kn) = O(n)$. This is because, $\#\text{edges} = \sum_{i=0}^{k-1}[E(L_i) + E(L_i, L_{i+1})] \leq \sum_{i=0}^{k-1}[4k|L_i| + 4k(|L_i| + |L_{i+1}|)] = 8k\sum_{i=0}^{k-1}|L_i| + 4k\sum_{i=0}^{k-1}|L_{i+1}| \leq 12kn = O(kn)$. Thus, we can find a C_{2k} that passes through v in time $O_k(n)$ using the algorithm from Lecture 2.

(b) We have $4k|L_i| < |E(L_i)|$ for some i. Assume that L_i is connected (otherwise, restrict to a connected component of L_i satisfying this condition). Check if L_i is bipartite (this can be done in $O(n)$ time using BFS) and consider the two cases:

   (i) L_i is not bipartite. We ascend to the lowest common ancestor (LCA) a of the vertices of L_i in the (partial) BFS tree. By definition, at least two of the children of a are ancestors of vertices in L_i. We arbitrarily pick one of them, and color its descendants in L_i in red. The remaining vertices in L_i are colored with blue. Denote the distance of a from L_i by d. By the Lemma above, since $E(L_i) > 4k|L_i| = 2 \cdot (2k) \cdot |L_i|$, we can find a path of length $2k - 2d$ between two differently-colored vertices in L_i in time $O(|E(L_i)|) = O(n)$. The concatenation of this path with the disjoint paths from its endpoints to a forms a C_{2k}.

(ii) L_i is bipartite, $L_i = A \uplus B$. This time, find the LCA a of A only. Color the vertices of A in red and blue as before, while the vertices in B are colored with green. By the Lemma above, there exists a path of length $2k-2d$ connecting two differently-colored vertices. Since $2k - 2d$ is even and L_i is bipartite, the path's endpoints lie on the same side. Since B is monochromatic (green), the two endpoints must both lie in A, and we are done as in the previous case.

(c) By BFS structure, the edges between L_i and L_{i+1} form a bipartite graph with sides L_i and L_{i+1}. We apply the same argument as in case (b)(ii) with $A = L_i$ and $B = L_{i+1}$.

□

Theorem 3.8 (Bondy-Simonovits [4])**.** $\mathrm{ex}(C_{2k}, n) = O(kn^{1+1/k})$.

Proof. Let G be a graph with $\geq 100kn^{1+1/k}$ edges, and assume by contradiction that G contains no C_{2k}. Then our algorithm fails to find a C_{2k} in G, which can happen only when we terminate due to condition (a); we shall prove that this is impossible.

G has a subgraph of minimum degree at least $100kn^{1/k}$, so for simplicity we assume the minimum degree condition holds for G itself. For condition (a) to occur, we must have fully discovered layers $L_0, \ldots, L_{k-1}$ while conditions (b),(c) remained unsatisfied. That is, for every $1 \leq i \leq k-1$:

$$4k|L_i| \geq |E(L_i)|$$
$$4k(|L_i| + |L_{i+1}|) \geq |E(L_i, L_{i+1})|$$
$$4k(|L_{i-1}| + |L_i|) \geq |E(L_{i-1}, L_i)|$$

The right-hand sides sum up to the total number of edges touching L_i. The sum of the left-hand sides provides an upper bound for that value. We also bound it from below using the minimum degree. Overall,

$$12k(|L_i| + |L_{i-1}| + |L_{i+1}|) \geq |E(L_i)| + |E(L_i, L_{i+1})| + |E(L_{i-1}, L_i)| \geq |L_i| \cdot (100/2) \cdot kn^{1/k}$$

(dividing by 2 for doubly-counted internal edges). Hence

$$|L_i|+|L_{i-1}|+|L_{i+1}| \geq |L_i| \cdot \frac{50}{12} n^{1/k} \Rightarrow |L_{i+1}|+|L_{i-1}| \geq |L_i| \cdot (\frac{50}{12} n^{1/k} - 1) \geq |L_i| \cdot (4n^{1/k} - n^{1/k}) \Rightarrow$$

$$|L_{i+1}| + |L_{i-1}| \geq |L_i| \cdot 3n^{1/k}.$$

We have $|L_0| = 1$ and it follows by induction that for $i \geq 1$

$$|L_i| \geq 3n^{i/k}.$$

Implying $|L_k| \geq 3n^{k/k} = 3n$, a contradiction. □

3.4 Lower Bounds for Algorithms

Proving unconditional, nontrivial time complexity lower bounds for specific computational problems remains an open challenge. Consequently, we rely on two alternative methodologies:

1. Lower bounds under constrained computational models (e.g., comparison-based sorting).
2. Conditional lower bounds via complexity-theoretic hypotheses, established using computational reductions.

We shall focus on the latter approach.

3.4.1 Fine-Grained Complexity

Fine-grained complexity adapts classical reductions to distinguish between specific polynomial-time complexities. This requires both tighter reductions and stronger foundational hypotheses. Before stating some of the popular hypotheses, we recall some well-known facts regarding the SAT problem.

3.4.2 SAT

- For general SAT, no algorithm is currently known that beats $O^*(2^n)$ (where O^* suppresses polynomial factors).
- Recall: k-SAT asks if a Boolean formula in CNF, with exactly k literals per clause, is satisfiable.
  - $k = 2$: solvable in linear time.
  - $k \geq 3$: NP-Complete.
  - Algorithms exist with running time $(2 - \varepsilon_k)^n$ for some $\varepsilon_k > 0$. However, in all of them, $\varepsilon_k \xrightarrow[k\to\infty]{} 0$.[11, 16]

3.4.3 Common Hypotheses

1. **ETH (Exponential Time Hypothesis):** SAT takes $\Omega(C^n)$ time for some $C > 1$.
2. **SETH (Strong ETH):**[12] $\forall \varepsilon > 0\ \exists k$ such that k-SAT $\notin$ Time$((2 - \varepsilon)^n)$.
3. **APSP Conjecture:** APSP $\notin$ Time$(n^{3-\varepsilon})$ for any $\varepsilon > 0$.
4. **Triangle Detection Conjecture:**
   - Weak version: cannot be solved in time $m^{1+o(1)}$.
   - Strong version: cannot be solved in time $n^{2-\varepsilon}$ for any $\varepsilon > 0$ (even when $m \leq n^{3/2}$).

3.4.4 Reductions from SETH

Orthogonal Vectors (OV)

- **Input:** Two sets $A, B \subseteq \{0,1\}^d$, where $|A| = |B| = n$. We often assume that $d =$ polylog(n).

- **Output:** Determine whether there exist $a \in A$, $b \in B$ that are orthogonal over $\mathbb{Z}$, i.e.

$$\langle a, b \rangle = \sum_{i=1}^{d} a_i b_i = 0.$$

- **Algorithms:**

  1. Iterate over all n^2 pairs; $O(n^2 d)$ time.
  2. Insert set A into a hash table. For each vector in B, iterate over all its orthogonal vectors and check whether any of them exist in the table; $O(2^d n)$ time.

Theorem 3.9. *When $d = \text{polylog}(n)$, SETH $\Rightarrow$ OV $\notin \text{Time}(n^{2-\varepsilon})$ for any $\varepsilon > 0$.*

Proof. Assume there is an algorithm $\mathcal{A}$ for OV running in time $O(n^{2-\varepsilon})$. We reduce k-SAT to OV: given a k-CNF formula φ with N variables and M clauses, we produce an OV instance that $\mathcal{A}$ solves. Divide the variables into two sets:

$$X = \{x_1, \ldots, x_{N/2}\}, \quad Y = \{x_{N/2+1}, \ldots, x_N\}.$$

We construct two sets of vectors $A, B \subseteq \{0,1\}^M$. For each assignment to the variables in X we add a vector to A, whose i-th coordinate is set to 0 iff the partial assignment satisfies clause i. We do the same for Y, B. Notice that

$$n := |A| = |B| = 2^{N/2} \text{ and } d := M = O(N^k) = O(\log^k n),$$

so the OV instance has $d = \text{polylog}(n)$ as required. Since $n = 2^{N/2}$, algorithm $\mathcal{A}$ runs in time $O(n^{2-\varepsilon}) = O(2^{(1-\varepsilon/2)N})$, contradicting SETH. It remains to show correctness:

$$\exists a \in A, b \in B \text{ s.t. } \langle a, b \rangle = 0 \iff \varphi \in \text{SAT}$$

($\Rightarrow$) a, b correspond to assignments ρ_X, ρ_Y for the variables in X, Y respectively. Since every coordinate is 0 either in a or in b, each clause is satisfied by either ρ_X or ρ_Y. Hence, the combination of ρ_X, ρ_Y defines a satisfying assignment ρ to all the variables.

($\Leftarrow$) Let ρ be a satisfying assignment for φ. ρ induces partial assignments ρ_X, ρ_Y for the variables in X, Y respectively, which define vectors $a \in A, b \in B$. In each clause there is at least one literal which ρ sets to true; its underlying variable is either in X or in Y. That is, for each clause i, at least one of ρ_X, ρ_Y satisfies it, hence $a_i = 0$ or $b_i = 0$. Therefore, $\langle a, b \rangle = 0$. □

Remark: The *Sparsification Lemma*[12] states that any k-CNF formula is equisatisfiable to a formula with $c_k \cdot n$ clauses.

Diameter

- **Input**: An undirected graph G.
- **Output**: The longest shortest path in the graph,

$$\max_{u,v \in V} d_G(u, v).$$

- **Algorithm:** Run BFS from each vertex; assuming $m = n^{1+o(1)}$, it takes $O(n^{2+o(1)})$ time.

Theorem 3.10. *When* $m \leq n^{1+o(1)}$, SETH $\Rightarrow$ Diameter $\notin$ Time$(n^{2-\varepsilon})$ *for any* $\varepsilon > 0$.

Proof. By reduction to OV (homework). □

Chapter 4

Fine-Grained Complexity and Cycle Reductions

Date: May 11, 2026
Scribe: Naomi Green

4.1 Reminder: Diameter Lower Bounds

Assuming SETH (or OV, as it is implied by SETH), one cannot compute the diameter of a graph in time n^{2-}, even when $m = \widetilde{O}(n)$.

The reduction is from OV. Given two sets of vectors A, B, we want to distinguish between: $\exists a \in A,\ b \in B$ such that $\langle a, b\rangle = 0$, and the case where no such pair exists.

The basic idea is to build a graph in which, for $a \in A$ and $b \in B$, $d(a,b) = \begin{cases} 2, & \langle a,b\rangle > 0, \\ > 2, & \langle a,b\rangle = 0. \end{cases}$
We do this by creating a graph with vertices corresponding to $A \cup B \cup [d]$, and drawing an edge from $a \in A$ or $b \in B$ to $i \in [d]$ if the i'th bit in the vector (a or b) is 1. There is a small issue: the graph also contains coordinate vertices (corresponding to $[d]$), so we need to ensure that the diameter is controlled not only between vertices in A and B, but also when one of the vertices is a coordinate vertex. This is fixed by adding two auxiliary vertices x, y, connected to $A \cup [d]$ and $B \cup [d]$ respectively. The number of vertices in the construction is $2n + poly\log(n) + 2 = O(n)$. The number of edges in the construction is $2n \cdot poly\log(n) + 2n + poly\log(n) + 1 = \widetilde{O}(n)$. The final construction appears in Figure 4.1.

After this fix, the distance is never more than 3, even with the coordinate vertices. Moreover, $Diam(G) = 2 \iff$ there are no orthogonal pairs in A, B, because in this case for every $a \in A, b \in B$ exists i such that $a_i = b_i = 1$, which means the construction has the edges $\{a,i\},\{b,i\}$. If there exists an orthogonal pair, then $Diam(G) = 3$. Thus, distinguishing between diameter 2 and diameter 3 is enough. In particular, a $(\frac{3}{2}-)$-approximation for diameter would already distinguish the two cases. Note that this reduction does not naturally give a lower bound for algorithms that give a rougher approximation.

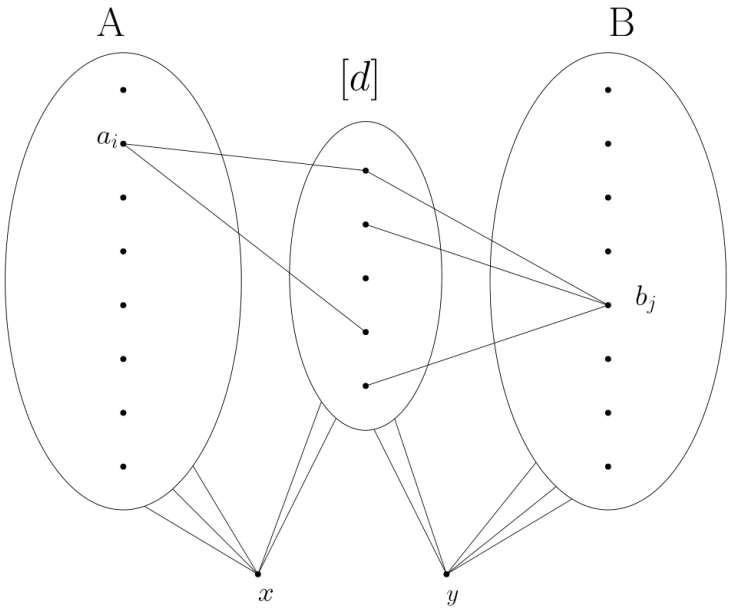

Figure 4.1: Diameter to OV reduction

4.2 Cycle Finding Algorithms

We previously saw algorithms for finding cycles:

- C_3 in $O(m^{2\omega/(\omega+1)})$ time
- C_4 in $O(m^{4/3})$ time
- C_{2k} in $O_k(n^2)$ time
- C_{2k+1} in $O_k(n^\omega)$ time, which becomes $O_k(n^2)$ if $\omega = 2$

A natural question that arises from these results is, can we find C_k in near-linear time, meaning $O(m^{1+o(1)})$? Or – if we are willing to assume that triangle finding is hard, can we prove that finding C_k is hard for other values of k?

4.3 The All-Edges Triangle Assumption

We shall assume the fine-grained complexity assumption on the hardness of the following problem.

All-Edges Triangle. Given a graph G, determine for every edge whether it is contained in some triangle.

Williams and Xu [23] showed that if APSP is hard (cannot be computed in subcubic time), or if 3SUM is hard (cannot be computed in subquadratic time), then All-Edges Triangle cannot be solved in time n^{2-}, even when $\Delta \leq \sqrt{n}$ (where Δ denotes the maximum degree in the graph), and the graph contains at most $n^{1.6}$ triangles. This final promise is non-trivial as even under $\Delta \leq \sqrt{n}$, there may be n^2 triangles in the graph.

4.4 Reductions Between Cycle-Finding Problems

4.4.1 Odd cycles

Claim 4.1. *For every constant k, if there is an $O(m^{1+o(1)})$-time algorithm for finding C_{2k+1}, then there is also an $O(m^{1+o(1)})$-time algorithm for finding C_3.*

Proof. First, we reduce triangle finding to triangle finding in a tripartite graph.

Lemma 4.2. *Triangle finding is equivalent, up to near-linear reductions, to triangle finding in tripartite graphs.*

Proof. One direction is trivial, as tripartite graphs are a special case of general graphs.

For the other direction, let G be a general graph, and color each vertex independently and uniformly with one of three colors. Given such a coloring, construct a tripartite graph by taking the three color classes as the parts and deleting all edges whose endpoints have the same color.

If G is triangle-free, then the resulting tripartite graph is also triangle-free. On the other hand, if G contains a triangle, then any fixed triangle is properly colored with constant probability. In that case, all three of its edges remain in the tripartite graph. Therefore, by repeating the construction for $O(\log n)$ independent colorings, with high probability at least one of the resulting tripartite graphs contains a triangle. Thus, triangle finding in general graphs reduces to triangle finding in tripartite graphs with only an $O(\log n)$ overhead. □

Now let G be tripartite, with parts V_1, V_2, V_3. We construct a graph G by replacing every edge between V_1 and V_2 by a path of length $2k-1$. Edges between V_1, V_3 and between V_2, V_3 remain unchanged. The resulting G is depicted in Figure 4.2. We claim that $C_3 \subseteq G \iff C_{2k+1} \subseteq G$.

Indeed, every triangle in the tripartite graph has exactly one edge between V_1 and V_2. Replacing that edge by a path of length $2k-1$ turns the triangle into a cycle of length $(2k-1)+2 = 2k+1$.

Conversely, suppose G contains a C_{2k+1}. The graph is layered as $V_1,\ U_1, \ldots, U_{2k-2},\ V_2,\ V_3$, where the U_i's are the internal layers of the subdivided V_1–V_2 edges. Removing any one layer leaves a bipartite graph. This is because each layer is an independent-set and there are $2k+1$ layers, so removing one layer leaves us with $2k$, an even number of layers. The remaining structure of layers degenerates from a continuous cycle into a path of layers or disconnected components, and we can construct a valid 2-coloring by partitioning the remaining layers into two disjoint sets. Therefore, an odd cycle must use all layers. Since the cycle has length exactly $2k+1$, and it must visit all $2k+1$ layers, it visits each layer exactly once. Hence it uses exactly one subdivided V_1–V_2 path, together with one edge from V_1 to V_3 and one edge from V_3 to V_2. Contracting the subdivided path gives a triangle in G.

□

4.4.2 From $C_{k\ell}$ to C_k

Claim 4.3. *If we can find $C_{k\ell}$ in near-linear time $O(m^{1+o(1)})$, then we can find C_k in near-linear time $O(m^{1+o(1)})$.*

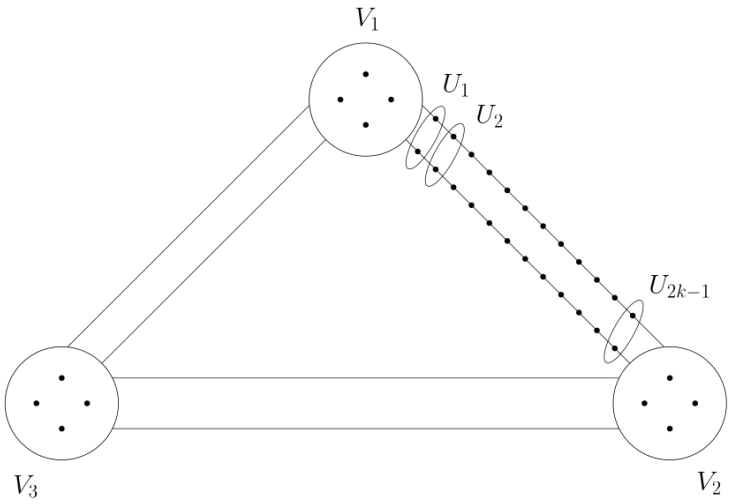

Figure 4.2: C_3 to C_{2k+1} reduction.

Proof. Given G, construct G by replacing every edge of G by a path of length ℓ. Clearly, every C_k in G becomes a $C_{k\ell}$ in G. Conversely, every edge of G lies on one of the replacement paths. If a cycle in G uses an internal vertex of one such path, then it must use the entire path. Thus every cycle in G is obtained by subdividing a cycle of G. Since each original edge contributes exactly ℓ edges, a cycle of length $k\ell$ in G contracts to a cycle of length k in G. □

4.4.3 Removing Short Cycles

By what we've seen so far, if triangle finding is hard in near-linear time, then finding C_k is hard whenever k has an odd factor. Indeed, if k is not a power of 2, then $k = 2^r q$ for some odd $q > 1$, and hardness of triangle finding implies hardness of C_q finding (by Claim 4.1), which implies hardness of C_k finding (by Claim 4.3).

The missing case is when k is a power of 2, thus it remains to show that hardness of triangle finding implies hardness of C_4 finding.

The common obstacle to making the above reductions work for C_4's is that the input graph may already contain many short cycles. Suppose we were promised that triangle finding is hard even in C_4-free graphs, or more generally in graphs with no short cycles. Then the simple reductions above would work.

So the goal is to reduce an input graph G to a graph G such that:

$$C_3 \subseteq G \qquad \iff \qquad C_3 \subseteq G,$$

and G has no cycles of lengths $4, \ldots, k$.

We show an easier version that is enough for many use cases: instead of removing all short cycles, we leave very few of them.

We focus on graphs with maximum degree at most $\sqrt{n}$, because the reduction starts from All-Edges Triangle instances. Generally, this means $\#C_k \leq n(\sqrt{n})^{k-1} = n^{(k+1)/2}$. Our goal will be to leave at most n^{2-} such cycles.

Theorem 4.4 (Abboud–Bringmann–Khoury–Zamir [1]). *For every constant $k \geq 4$, every $\alpha \in (0, 1/k)$, every $\in (0, \frac{3-\omega}{4})$, and every graph G with n vertices and maximum degree at most $\sqrt{n}$,*

one can compute, in time n^{2-}, a set of edges E' and subgraphs $G_1, \ldots, G_s$ with $s = n^{3/2-3\alpha}$, such that:

1. *Every edge in E' is contained in a triangle in G.*
2. *Every edge that is contained in a triangle in G is either in E', or is contained in a triangle in some G_i.*
3. *With high probability, each G_i has $O(n^{1/2+\alpha})$ vertices and maximum degree $O(n^{\alpha})$. Which means the number of edges in all the subgraphs is $\leq n^{3/2-3\alpha} \cdot n^{1/2+\alpha} \cdot n^{\alpha} = O(n^{2-\alpha})$.*
4. *For every i, the expected total number of $C_{k'}$'s of lengths $4 \leq k' \leq k$ in G_i is at most*

$$O\left(n^{\frac{\omega-1}{4}+k\alpha+}\right).$$

Which means the number of all $C_{k'}$ cycles in the subgraphs is

$$\leq O\left(n^{\frac{\omega-1}{4}+k\alpha+}\right) \cdot n^{3/2-3\alpha} \approx O\left(n^{\frac{\omega-1+6}{4}}\right) = O\left(n^{2-'}\right) \text{ since } \omega < 3$$

4.4.4 Using the Theorem for C_4 Finding Hardness

Claim 4.5. *Assuming All-Edges Triangle hardness, there is no $O\left(m^{1+o(1)} + tm^{o(1)}\right)$-time algorithm that, given a graph, finds t different copies of C_4 or reports that fewer than t copies exist.*

Recall that in time $O(n^2+t)$ we know how to do this by hashing P_2's and checking for collisions in the hash tables (similarly to a C_4 finding algorithm we've seen). The point is to rule out near-linear time.

Proof. Let G be a graph meeting the requirements of All-Edges Triangle. Assuming by way of contradiction the existence of the above algorithm, we will answer All-Edges Triangle on G in $O(n^{2-})$ time, reaching the desired contradiction.

Apply Theorem 4.4 to obtain E' and subgraphs $G_1, \ldots, G_s$. The edges in E' can already be answered, because every edge in E' is in a triangle.

For each G_i: first make G_i tripartite (by applying a random coloring) with parts V_1, V_2, V_3. Then replace every $V_1 - V_2$ edge by a P_2 so that every triangle in G_i becomes a C_4 in the new graph G_i.

Now use the assumed fast algorithm for listing *all* C_4's on each G_i.

Every triangle in G_i gives a C_4 in G_i, and each listed C_4 can be checked to see whether it came from such a triangle or was created by the reduction. Thus by finding all relevant C_4's, we recover all triangle edges in the G_i's. Together with E', this lets us solve All-Edges Triangle.

The cost of the reduction and edge blow-up is n^{2-}. It remains to find the cost of running the C_4-listing algorithm. The $m^{1+o(1)}$ part of the cost sums to $n^{2-\alpha}$ (as that bounds the number of edges in all G_i's together). The $tm^{o(1)}$ cost is proportional to the number of C_4's that must be listed. These are either C_3's in G (of which there are $< n^{1.6}$, by our assumptions), or C_4's in G_i (of which there are $< n^{2-\alpha'}$ for some constant α'). Finally, testing whether a C_4 in G_i

came from a triangle in G_i takes linear time in the number of listed C_4's. Overall, the cost is $O(n^{2-})$, contradicting All-Edges Triangle hardness. □

Exercise 4.6 (distance oracles)**.** *Show that, assuming All-Edges Triangle hardness, one cannot build a data structure with initialization time $O(m^{1+o(1)})$ that answers every query of the form "what is the distance between u and v?" up to a k-approximation in time $m^{o(1)}$.*

4.4.5 Proof for Theorem 4.4

Proof. Assume without loss of generality that G is tripartite (similarly to Lemma 4.2). Randomly partition every color class into $n^{1/2-\alpha}$ subsets of size $n^{1/2+\alpha}$. Every choice of 3 parts, one from each color class, forms a G_i. This meets requirement 2 of Theorem 4.4, as every C_3 is in the G_i that contains the parts corresponding to this C_3. This construction gives $s = (n^{1/2-\alpha})^3$ and $|V(G_i)| = 3n^{1/2+\alpha}$, thus meeting requirement 3.

What happens to $\deg(v)$ in G_i that contains v? In expectation, its degree into some part in a different color is $\frac{\sqrt{n}}{n^{1/2-\alpha}} = n^\alpha$, and is distributed as, or dominated by, a $\mathrm{Bin}(\sqrt{n}, n^{\alpha-1/2})$ random variable. Thus, the probability that $\deg(v) > 2n^\alpha$ is $< \exp(-\Theta(n^\alpha))$. By a union bound, all vertices in all G_i's have a maximum degree $O(n^\alpha)$, meeting requirement 3.

Requirement 4, which we show for $k = 4$; the proof is similar for $k' > 4$: every C_4 must contain two vertices of the same color class, thus it survives in some G_i in probability at most $1/n^{1/2-\alpha}$, as this is the probability of two vertices with the same color being in the same random part. Thus, in expectation, the total number over all G_i's is at most $\#C_4(G)\frac{n^\alpha}{\sqrt{n}}$, which could, in the worst case, be $n^{2+\alpha}$. The new goal is to reach a situation where $\#C_4(G) \le n^{2-\Omega(1)}$, possibly at the cost of moving some edges to E'.

The intuition is: if there are many C_4's, then there is a dense subgraph. Inside such a dense subgraph, it is worthwhile to solve All-Edges Triangle directly.

Definition 4.7. *Let $\gamma = \frac{\omega-1}{4}+ < 1/2$. A subgraph with at most $2\sqrt{n}$ vertices is called* dense *if it has at least $n^{1/2+\gamma}$ edges.*

Definition 4.8. *An edge e is called* dense *if it participates in at least $n^{1/2+\gamma}$ C_4's.*

The plan is:

1. If there are "many" C_4's, then there are "many" dense edges.
2. Given an edge e, we can "quickly" test whether it is dense, and if e is dense, we can "quickly" find a dense subgraph.
3. On dense subgraphs, we can solve all relevant triangle questions "fast enough".

Indeed,

1. Denoting by $\#C_4(e)$ the number of C_4's that e participates in, and by t the number of dense edges, we obtain

$$\#C_4 \le \sum_e \#C_4(e) \le \sum_{e \text{ is dense}} \#C_4(e) + \sum_{e \text{ is not dense}} \#C_4(e) \le t(\sqrt{n})^2 + n^{3/2}n^{1/2+\gamma}.$$

The $(\sqrt{n})^2$ term is as e has at most $\sqrt{n}$ neighbors at each side, and a C_4 is only obtained by some pair of these neighbors having an edge between them. The number of such pairs is at most $(\sqrt{n})^2$. Thus, $t \geq \frac{\#C_4 - n^{2+\gamma}}{n}$, so if $\#C_4 > 2n^{2+\gamma}$, $t > n^{1+\gamma}$. In particular, assuming that $\#C_4 > 2n^{2+\gamma}$ implies that a randomly chosen edge is dense with probability $\geq \frac{n^{1+\gamma}}{n^{3/2}} = n^{\gamma-1/2}$.

2. For an edge $e = (u, v)$, the number of C_4's containing e is exactly the number of edges between $N(u)$ and $N(v)$. Thus, e is dense if and only if the bipartite graph between $N(u)$ and $N(v)$ is dense (see Figure 4.3). So to check if e is dense, randomly sample pairs from $N(u) \times N(v)$ and check whether they are edges. If there are $> n^{1/2+\gamma}$ such edges, then in $n^{1/2-\gamma}$ tests, we expect to find an edge. Thus, after $10 \log n \cdot n^{1/2-\gamma}$ such tests, we'll know with probability $> 1 - 1/n^{10}$ to distinguish between $> 2n^{1/2+\gamma}$ edges and $< n^{1/2+\gamma}$ edges. In particular, we find a dense subgraph in time $O(\log n \cdot n^{1-2\gamma})$ (the time it takes to check if an edge is dense, times the number of expected trials until we indeed find such an edge).

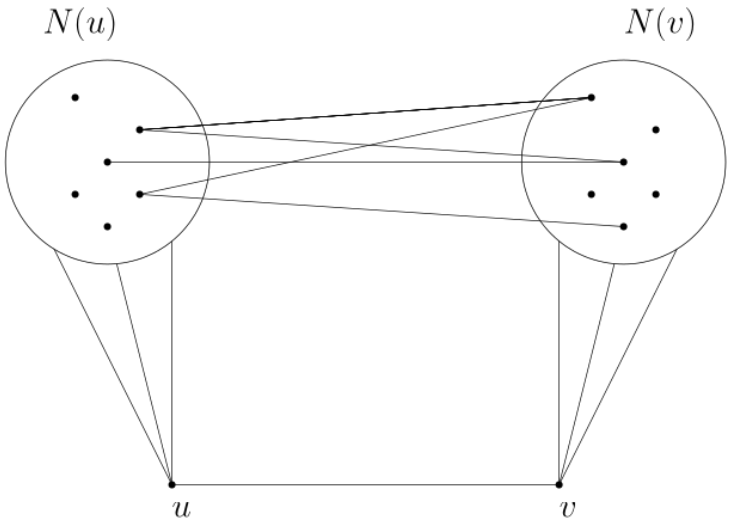

Figure 4.3: The neighborhood of a dense edge.

3. We provide a proof sketch for this final part. We want to "get rid" of all $n^{1/2+\gamma}$ edges from the dense subgraph (and answer All-Edges Triangle accordingly), and repeat this procedure until no dense subgraphs remain. We pay $n^{1-2\gamma}$ to delete $n^{1/2+\gamma}$ edges, thus every deleted edge "costs" $n^{1/2-3\gamma}$, so all edge deletions cost $\leq n^{3/2} n^{1/2-3\gamma} = n^{2-3\gamma} \ll n^2$.

   Let H be a dense subgraph. To delete it we need to know, for every edge of H, whether it is contained in a triangle in the original graph G.

   For some threshold T, partition the vertices of G into

$$V_{\text{low}} = \{v \in V : \deg(v) \leq T\}, \qquad V_{\text{high}} = \{v \in V : \deg(v) > T\}.$$

   For triangles using a low-degree vertex, we can check all pairs of neighbors of that vertex.

   For triangles involving only high-degree vertices, we use the fact that there cannot be too many high-degree vertices. We then run a fast matrix multiplication based triangle-finding algorithm on the subgraph induced by V_{high} (similarly to a homework problem from week 1). This lets us process the dense subgraph quickly enough, delete it, and continue until the remaining graph has few short cycles.

□

Chapter 5

Property Testing

At the beginning of the lesson we finished a discussion from the previous lesson.

At the end of the previous lesson, we wanted to show that if there is a graph G with maximum degree $\sqrt{n}$, we can keep $\geq n^{2-\varepsilon}$ copies of C_4 without "harming" the triangles, in time $n^{2-\varepsilon}$.

1. We saw we can keep approximately $\frac{n^\varepsilon}{\sqrt{n}} \# C_4(G)$.
2. Therefore, we wanted to show that G can be transformed into G' with $n^{2.49}$ copies of C_4.
   - We looked for subgraphs of size $\sqrt{n}$ vertices and $n^{\frac{1}{2}+\gamma}$ edges (where $\gamma = \varepsilon + \frac{\omega+1}{4}$).
   - We noticed that finding such subgraphs effectively "pays for itself".
   - It remains to show that finding all edges in G that are in a triangle utilizing edges from the dense subgraph H is also fast enough.

Let's take the graph G, which contains the dense subgraph H. Triangles have one vertex $v \in V$ (which may or may not be in H) and one edge originating from H. We partitioned all of V into sets V_ℓ and V_h based on whether the number of neighbors of v inside H is $\leq$ or $>$ than $n^{\frac{1}{2}-\beta}$.

For V_ℓ: A vertex of degree d in H can be processed in $O(d^2)$ time (by iterating over all pairs of its neighbors). So "in total", the time taken is $\frac{n}{d} \cdot d^2 = nd$ where n is the number of edges touching H. Formally, we divide V_ℓ into subsets of vertices with degrees in $[1,2), [2,4), \ldots, [\frac{1}{2}n^{\frac{1}{2}-\beta}, n^{\frac{1}{2}-\beta})$. Then processing V_ℓ costs at most $n \cdot n^{\frac{1}{2}-\beta} = n^{\frac{3}{2}-\beta}$ time.

For V_h: The number of such vertices is at most $\frac{n}{n^{\frac{1}{2}-\beta}} = n^{\frac{1}{2}+\beta}$. We partition V_h into n^β disjoint subsets of size $\sqrt{n}$ each, and then we find a triangle with H using matrix multiplication in time $n^\beta \cdot (n^{\frac{1}{2}})^\omega = n^{\frac{\omega}{2}+\beta}$.

We balance the two time expressions by setting $\frac{\omega}{2} + \beta = \frac{3}{2} - \beta$. This gives $\beta = \frac{3-\omega}{4}$. Thus, the running time is $n^{\frac{\omega}{2}+\frac{3-\omega}{4}} = n^{\frac{3+\omega}{4}}$.

How many times did this process occur? Each time we deleted $n^{\frac{1}{2}+\gamma}$ edges. There were $n^{\frac{3}{2}}$ edges initially, so it occurred at least $n^{1-\gamma}$ times. Total cost:

$$n^{\frac{3+\omega}{4}+1-\gamma} = n^{\frac{3+\omega}{4}+1-\frac{\omega+1}{4}-\varepsilon} = n^{2-\varepsilon}$$

5.1 Introduction

When can we solve a problem without reading all of the input? We've already seen some examples:

- We've seen we can find P_t in $O(n)$ rather than $O(m)$.
- In the previous lesson, we've seen we can test graph density.

To clarify, we assume a computational model where we have random access to the input and can sample any part of it.[1]

Our goal is to only perform $O(1)$ samples (queries) with respect to the input size. The "price" for doing so is that we'll have to use randomization, and we can only solve gap problems. That is, our algorithm will work if we only need to distinguish between inputs for which the answer is *yes* and inputs for which the answer is a *"hard" no*. It might give an arbitrary answer on inputs for which the answer is *no* but are not "far enough" from the *yes* inputs.

This gap will often be expressed using a parameter ε. We will say that we distinguish between "yes" inputs and "no" inputs that are ε-*far* from the "yes" inputs. The number of samples will usually depend on ε.

5.2 A Basic Example

Suppose we have a binary vector $v \in \{0,1\}^n$, and we have random access to its entries. We want to solve Majority for it. That is, to decide whether $\text{weight}(v) = \#\{i \in [n] \mid v_i = 1\} \geq \frac{n}{2}$.

If $\text{Maj}(v) = 1$ (the answer is "yes"), we want to answer "yes", and if we are (at least) ε-far from $\text{Maj}(v) = 1$ (that is, $\text{weight}(v) < \left(\frac{1}{2} - \varepsilon\right) n$), we want to answer "no" with probability $> 90\%$. Otherwise, we don't care what the answer is.

Solution. Sample k indices and answer "yes" if at least a fraction of $\frac{1}{2} - \frac{\varepsilon}{2}$ of them are 1s (and otherwise answer "no").
What should k be? Every sample is distributed Bernoulli(p) where $p = \frac{1}{n}\text{weight}(v)$. Thus, the expectation of the sum of all samples is pk and the variance is $p(1-p)k = \Theta(pk)$. Therefore, the standard deviation is $\Theta(\sqrt{pk})$.
So, we need to have roughly $\sqrt{pk} < \frac{1}{10} \cdot \frac{\varepsilon}{2} k$ (with $\frac{1}{10}$ being an arbitrary positive constant that is sufficiently small). It follows that $k = \Omega\left(\frac{1}{\varepsilon^2}\right)$.

Claim 5.1. *Every deterministic algorithm requires $\Omega(n)$ queries.*

Proof. Every algorithm looks at the outputs of the previous queries and decides whether to produce an output or perform an additional query. If the algorithm is deterministic then so are the decisions.

Suppose by way of contradiction that the number of queries is always $\leq \left(\frac{1}{2} - \varepsilon\right) n$. Consider the execution of the algorithm on the input $\vec{v} = \vec{1}$. This is a "yes" input, so the algorithm would

[1] If we work in the traditional Turing machine model with sequential access, it's not difficult to see we need to perform $\Omega(n)$ steps anyway.

read a set $I \subseteq [n]$ of $\leq \left(\frac{1}{2} - \varepsilon\right) n$ indices and output "yes". Therefore, if we look at an input $\vec{u}$ where for every index $i \in [n]$,

$$u_i = \begin{cases} 1 & i \in I \\ 0 & i \notin I \end{cases}$$

we get that the algorithm must output "yes" on $\vec{u}$ as well, but it should output "no". □

Claim 5.2. *If $\varepsilon = 0$, then even a randomized algorithm requires $\Omega(n)$ queries in expectation.*

Proof sketch. The idea is to construct two distributions of inputs that are very close (making it hard to find a query that distinguishes between them), but for which the correct outputs differ. We draw a vector v with exactly $\lceil \frac{n}{2} \rceil$ ones. Then, we uniformly pick an index i where $v_i = 1$, and with probability $\frac{1}{2}$ we flip v_i to 0. If the algorithm samples $\ll \frac{n}{4}$ indices, it will highly likely miss the index i, and thus won't know what the correct answer is. □

One can prove this for a general $\varepsilon = o(1)$. In that case, we'll get a lower bound of $\Omega\left(\frac{1}{\varepsilon}\right)$ samples.

5.3 Graph Property Testing

Following Goldreich, Goldwasser, and Ron [10], we wish to distinguish between two cases for an input graph G:

- Does G have a specific property P ("yes").
- Do we need to modify at least εn^2 of its potential edges to make it satisfy property P ("no").

This restricts the properties we test to those that do not inherently constrain the graph's density. For instance, it is *not interesting* to test for the property $P =$ "contains C_4", since one would only need to change at most 4 edges to introduce a C_4. Similarly, testing $P =$ "does not contain C_4" is uninteresting (one could simply sample to see if the number of edges is $> \frac{\varepsilon}{2} n^2$; if so we say "no", and if not, we can delete $\leq \frac{\varepsilon}{2} n^2$ edges to make it C4-free, so the answer is "yes" as it's not ε far from having P).

What properties are interesting to check?

- "Does not contain C_3" (Triangle-free) or any non-bipartite graph H.
- "Is bipartite / ε-far from being bipartite", "k-colorable / ε-far".

Goal: Distinguish between G being bipartite and G being ε-far from bipartite (meaning that in any 2-coloring, there are $\geq \varepsilon n^2$ monochromatic edges).

We will show that the *canonical algorithm*[2] works. If G is 2-colorable, then any subgraph is also 2-colorable. Thus, on a "yes" instance, we will **always** answer "yes".

Theorem 5.3. *If G is ε-far from being bipartite, and we sample $\frac{1000 \lg(1/\varepsilon)}{\varepsilon^2}$ vertices, then with probability $> 90\%$, the induced subgraph on them is not 2-colorable.*

[2] which is: sample a random subgraph and check if the property holds on it.

Proof. We can "imagine" that we first sample a set U of $\frac{50\lg(1/\varepsilon)}{\varepsilon}$ random vertices along with all edges between them, and then another set T of $\frac{10|U|}{\varepsilon}$ random (potential) edges.

We say that the set U is "good" if there are at most $\frac{\varepsilon}{6}n$ vertices in G that have degree $\geq \frac{\varepsilon}{6}n$ and are not neighbors of any vertex in U.

Claim 5.4. *With probability $> 99\%$, the sampled set U is "good".*

Proof of claim. Suppose v is a vertex with degree $\geq \frac{\varepsilon}{6}n$. What is the probability that it does not have a neighbor in U? The probability that a random vertex is not a neighbor of v is $\leq 1 - \frac{\varepsilon}{6}$, so in total we get:

$$\left(1 - \frac{\varepsilon}{6}\right)^{\frac{50\lg(1/\varepsilon)}{\varepsilon}} \leq \exp\left(-\Theta\left(\lg\frac{1}{\varepsilon}\right)\right) \ll \varepsilon$$

Since the expected number of such "bad" vertices is $\ll \varepsilon n$, by Markov's inequality, the probability that there are $> \frac{\varepsilon}{6}n$ of them is $< 1\%$. Thus, with probability $> 99\%$, the number of bad vertices is at most $\frac{\varepsilon}{6}n$. □

Let us consider a 2-coloring $U_1 \uplus U_2$ **of** U and ask what it implies regarding the coloring of the rest of the graph: Except for the $\leq \frac{\varepsilon}{6}n$ "bad" vertices and the vertices of degree $< \frac{\varepsilon}{6}n$, the coloring (U_1, U_2) uniquely forces the colors of all remaining vertices in the graph, or it is immediately invalidated.

Because the bad vertices and those of low degree contribute at most $\frac{\varepsilon}{3}n^2$ edges in total, there is still a distance of at least $\frac{2}{3}\varepsilon n^2$ between any coloring of the rest of the graph and a valid bipartite coloring. This means that the coloring induced by (U_1, U_2) *on the entire graph* must contain at least $\frac{2}{3}\varepsilon n^2$ monochromatic edges.

This implies that if we were to sample $\Omega\left(\frac{1}{\varepsilon}\right)$ random (potential) edges from all of G, we would find a monochromatic edge and the coloring would be invalidated. Furthermore, if we sample our set T of $\frac{10|U|}{\varepsilon}$ edges, then the probability that this specific coloring is not invalidated is:

$$\left(1 - \frac{2}{3}\varepsilon\right)^{\frac{10|U|}{\varepsilon}} < 10^{-|U|}$$

But since U only has $2^{|U|}$ possible colorings, the union bound tells us that the probability that there exists *at least one* coloring of U that is not invalidated is:

$$\leq 2^{|U|} \cdot 10^{-|U|} = 5^{-|U|} \ll 1\%$$

Therefore, with high probability, no valid 2-coloring exists for the sampled subgraph, meaning it is not bipartite. □

Remarks:

- The algorithm still simply checks whether the induced subgraph is 2-colorable. The algorithm runs in $\tilde{O}\left(\frac{1}{\varepsilon^4}\right)$ time (checking whether a graph is 2-colorable is linear).
- If we only cared about the number of queries/samples and not the running time, then $\tilde{O}\left(\frac{1}{\varepsilon^3}\right)$ samples would suffice.

- The best upper bound currently known is $\tilde{O}\left(\frac{1}{\varepsilon^2}\right)$, and the lower bound is $\tilde{\Omega}\left(\frac{1}{\varepsilon^{3/2}}\right)$.

Exercise: Design a Property Tester for the question "Is G a BiClique?" (A complete bipartite graph $K_{s,n-s}$ up to vertex permutation, for some s).

5.4 Testing Triangle-Freeness

Next, we discuss testing for H-freeness (i.e., the graph does not contain H as a subgraph). In this lecture we focus on $H = K_3$, but the approach generalizes to any non-bipartite H (following algorithms by Alon, Krivelevich, and Shapira, ∼2000).

Note that the dependence on ε, at least for a general H, cannot be too pleasant. If $H = K_k$ and $\varepsilon = \frac{1}{2n^2}$, the problem is exactly the same as finding a k-clique in the graph.

Algorithm. For some constant (but large) number of times, sample three vertices at random and check if they are a triangle. If no triangle is found answer "yes", otherwise answer "no".

The completeness (the "yes" case) of the algorithm is trivial.
For soundness (the "no" case, where we are ε-far from being triangle-free), we need to show that for every $\varepsilon > 0$ there exists $\delta > 0$ such that if G is is ε-far from being triangle-free then it contains at least $\delta \cdot n^3$ triangles. This is known as the *Triangle Removal Lemma*.

Central Tool: Szemerédi's Regularity Lemma [20].
Intuitively: Every sufficiently large graph is "close" to looking "random".

Definition 5.5. *A bipartite graph with sides A and B is called γ-regular if for every $A' \subseteq A$ and $B' \subseteq B$ such that $|A'| \geq \gamma|A|$ and $|B'| \geq \gamma|B|$, it holds that $\left|\frac{e(A',B')}{|A'|\cdot|B'|} - \frac{e(A,B)}{|A|\cdot|B|}\right| < \gamma$.*

Lemma 5.6 (Szemerédi's Regularity Lemma)**.** *For every $\gamma > 0$ and integer $\ell > 0$, there exists a constant $T = T(\gamma, \ell)$ such that for every graph $G = (V, E)$ on $n \geq T$ vertices there exist $\ell \leq t \leq T$ and disjoint sets $V_0, V_1, \ldots, V_{t-1}$ satisfying:*

1. *For every $0 \leq i \leq t$, $\lfloor \frac{n}{t} \rfloor \leq |V_i| \leq \lceil \frac{n}{t} \rceil$*
2. *For all but at most γt^2 pairs (i, j), the bipartite graph (V_i, V_j) is γ-regular.*

Remarks:

- T is a constant that does not depend on n, only on γ and ℓ.
- This constant T is astronomically large; it is bounded by a Tower function of a polynomial in $1/\gamma$ (where $\text{Tower}(0) = 1$, $\text{Tower}(i) = 2^{\text{Tower}(i-1)}$). Specifically, $T \approx \text{Tower}(\gamma^{-5})$.

Claim 5.7. *The Regularity Lemma $\implies$ the Triangle Removal Lemma.*

Proof. We set $\gamma = \frac{\varepsilon}{5}$ and $\ell = \left\lceil \frac{5}{\varepsilon} \right\rceil$, and obtain $T = T(\gamma, \ell) = \text{Tower}\left(\text{poly}\left(\frac{1}{\varepsilon}\right)\right)$ from the Regularity Lemma.

(If G has fewer than T vertices, we sample all of it and check if it contains a C_3. In such a case, the lemma isn't needed and we can check directly, so it is sufficient to prove the lemma for a large enough n.)

Otherwise, by the Regularity Lemma, there is a partition $V_0, V_1, \ldots, V_{t-1}$ where all but γt^2 pairs are γ-regular.
We delete all the following edges from the graph:

1. All edges inside any V_i.
2. All edges between a pair V_i, V_j that is not γ-regular.
3. All edges between a pair V_i, V_j with density $< 2\gamma$.

Let's ask how many edges we deleted:

1. In the first case, $\leq n \cdot \left\lceil \frac{n}{t} \right\rceil \leq \frac{\varepsilon}{5} n^2$ (since each vertex only looks at those in its own group).
2. In the second case, $\leq \gamma \cdot t^2 \cdot \left\lceil \frac{n}{t} \right\rceil^2 \leq \frac{\varepsilon}{5} n^2$.
3. In the third case, not more than the density (2γ): $\leq t^2 \cdot \left(\frac{n}{t}\right)^2 \cdot 2\gamma \leq \frac{2\varepsilon}{5} n^2$.

In total, $< 0.9\varepsilon n^2$ edges.

Now all pairs are regular, and every V_i, V_j that is not an empty subgraph is with density $\geq 2\gamma$.

The remaining graph must have a triangle (since we changed $< \varepsilon n^2$ edges). In fact, this triangle must be between distinct V_1, V_2, V_3 (WLOG) such that every pair between them is γ-regular and with density $\geq 2\gamma$.

We want to show that the number of triangles between V_1, V_2, V_3 is at least $\left(\frac{n}{t}\right)^3 \cdot \frac{\gamma^3}{10}$, as if the edges were truly distributed randomly.

Observation 5.8. *For almost every vertex $v \in V_1$, there are $\geq \gamma \cdot \frac{n}{t}$ neighbors in V_2 (and similarly in V_3).*

Proof. Let U be the set of vertices in V_1 that have fewer than $\gamma \cdot \frac{n}{t}$ neighbors in V_2. It must be of size $< \gamma \cdot |V_1|$, because otherwise by regularity:

$$\frac{e(U, V_2)}{|U| \cdot |V_2|} \geq 2\gamma - \gamma = \gamma$$

Therefore, $(1-2\gamma)|V_1|$ vertices from V_1 have both $\geq \gamma |V_2|$ neighbors in V_2 and $\geq \gamma |V_3|$ neighbors in V_3. □

For each such v, we look at its set of neighbors in V_2, denoted U_2, and its set of neighbors in V_3, denoted U_3. Then by regularity it follows that:

$$e(U_2, U_3) \geq \gamma \cdot |U_2| \cdot |U_3| \geq \gamma^3 \cdot \left(\frac{n}{t}\right)^2$$

And this is therefore the number of triangles containing v. So if we sum over all the $(1-2\gamma)|V_1|$ good v's, we get:

$$\delta \cdot n^3 \equiv \frac{n^3 \cdot \varepsilon^3}{100 \cdot \text{Tower}\left(\text{poly}\left(\frac{1}{\varepsilon}\right)\right)} \approx \frac{n^3 \gamma^3}{t^3} \approx (1-2\gamma) \cdot \frac{n}{t} \cdot \gamma^3 \cdot \left(\frac{n}{t}\right)^2$$

for a constant δ. □

Exercise: Prove the Triangle Removal Lemma for any graph H of a fixed size.

Chapter 6

Usage of Edge Contraction in Algorithms

Date: May 25, 2026
Scribe: Gabriel Ginzburg

6.1 Introduction

Today's lecture is about edge contraction and algorithms for minimal spanning trees and min cut, both old rephrased through this new language and new improved algorithms using it.

Definition 6.1. *Let G be an undirected graph, and let $\{u, v\} = e \in E$ be an edge in it. The graph G/e is the graph that results from contracting $\{u, v\}$ into a single vertex.*

Note 6.2. *We will sometimes also keep parallel edges or even self-loops on the newly contracted vertex.*

6.2 Minimum Spanning Trees (MST)

A past example of usage of edge contraction is in Minimum Spanning Trees.

Problem 6.3. *Given a connected and undirected graph G, find the spanning tree with a minimum total edge weight. The weights can be arbitrary real numbers (both positive and negative).*

Note 6.4. *We can assume that all the weights are distinct and the MST is unique. Otherwise, in the case of non-distinct weights, we can break ties for edges with equal weights using the edges' indices.*

6.2.1 Algorithms that we've seen before

- Kruskal's algorithm which takes $O(m \log n)$ (for sorting the edges) and $O(m\alpha(m, n))$, where $\alpha(m, n)$ is the inverse Ackermann function resulting from the Union-Find in the algorithm.
- Prim's algorithm which takes $O(m + n \log n)$ using Fibonacci Heaps.

Claim 6.5. *An edge e of graph G is part of its MST $\iff$ e is an edge of minimal weight in some cut (S, S^c) of the graph.*

Proof. Let $e = \{u, v\}$ be an edge in the MST T of G. We define the cut $V = S \cup S^c$ to be the connected components of $T \setminus \{e\}$. This is well-defined because u and v are disconnected after removal of e. Otherwise, there would be another path between them, which, together with e, would create a cycle in the tree T, a contradiction.

If the defined cut contains an edge e' with $w(e') < w(e)$, then $(T \setminus \{e\}) \cup \{e'\}$ is also a spanning tree of G as it is connected and has $n - 1$ edges, and with a total weight less than the total weight of T, which is a contradiction. Therefore, e has minimal weight in the cut (S, S^c).

On the other hand, let $e = \{u, v\}$ be an edge of minimum weight in the cut (S, S^c) where $u \in S, v \in S^c$. Let us look at the MST T of G and assume by contradiction that e is not in T. Since T is an MST, there exists a unique path in it from u to v. This path begins on one side of the cut (S, S^c) and ends on the other side of the cut. Therefore, it contains at least one edge e' that passes through the cut. If we take $(T \setminus \{e'\}) \cup \{e\}$, then it is also a spanning tree, because a connected graph with $n - 1$ edges is a tree, and it contains a path between any two vertices (there is also a path between the endpoints of the erased edge e'). However, by our assumption $w(e) < w(e')$. Thus, the new spanning tree has a total weight less than T, the MST of the graph, a contradiction. Therefore, the edge e is indeed part of the MST T. □

Claim 6.6. *Let G be a graph, and let e be an edge of the graph that is inside its MST T. Then, $T = \{e\} \cup MST(G/e)$.*

Proof. Let e' be some edge in $MST(G/e)$. From Claim 1, it is minimal in some cut of G/e, and also minimal in some cut of the graph G, and therefore it is in the MST of G.
Since G/e has $n-1$ vertices, its MST contains $n-2$ edges. By adding the edge e to $MST(G/e)$, we get $n - 1$ edges that all must be inside T, which means that we get T itself. □

Algorithm 6.7 (Kruskal (1956) [14])**.** *Without using the language of edge contractions:*

- *Sort the edges $e_1 < e_2 < \ldots < e_m$.*
- *Go over the sorted edges. For each edge, if it doesn't create a cycle in the partial MST built so far, add it to the current MST (this part is done using Union-Find).*

In the context of edge contraction:
As long as the graph has more than one vertex, we take the minimal edge that isn't a self-loop. This edge is minimal in any cut that contains it, so we add it to the partial MST (using Claim 1), and then we contract it (using Claim 2).

Algorithm 6.8 (Prim (1957) [17])**.** *We state it directly with edge contractions:*

- *Start with an arbitrary vertex u.*
- *While the graph has more than one vertex, take the minimal non-self-loop that touches u, which is itself a minimal edge in the cut $(\{u\}, V \setminus \{u\})$, add it to the partial MST, and contract it.*
- *An implementation using Fibonacci heaps gets $O(1)$ amortized time for Decrease-Key and Find-Min, and $O(\log n)$ amortized for Delete-Min.*

Algorithm 6.9 (Borůvka (1926) [5])**.** *While the graph has more than one vertex:*

- *Every vertex chooses the edge with a minimal weight that touches it (this edge is always minimal in some cut—therefore it's in the MST).*
- *Contract all the chosen edges (can be done using BFS divided into connected components relative to the partial MST).*

This algorithm takes $O(m \log n)$ time but is very easy to parallelize.
Note that since the size of every connected component is at least 2, after every iteration, the number of vertices shrinks by at least a factor of 2. Every iteration takes $O(m)$ time, and there are no more than $O(\log n)$ iterations, resulting in a running time of $O(m \log n)$.

6.2.2 Fredman-Tarjan Algorithm [8]

The running time of the algorithm will be $O(m\beta(m,n))$.

Definition 6.10. $\log^{(k)}(n) = \log\log\ldots\log(n)$, *taking* log k *times.*
$\log^{(0)}(n) = n$
$\log^{(k)}(n) = \log\log^{(k-1)}(n)$

Definition 6.11. $\log^*(n) = \min\{k \mid \log^{(k)}(n) \leq 2\}$
This is the inverse function of $Tower$, that is, $\log^*(Tower(n)) = n$.

Definition 6.12. $\beta(m,n) = \min\{k \mid \log^{(k)}(n) \leq \frac{2(m+1)}{n}\}$

In a connected graph G we have $m \geq n-1$, which means $\frac{2(m+1)}{n} \geq 2$; therefore, $\beta(m,n) \leq \log^*(n)$.

Moreover, if $m \geq n\log^{(k)}(n)$, then $\beta(m,n) \leq k$.

Algorithm 6.13. *We will see an iteration that gets a parameter k and a graph G, runs in $O(m \log k)$ time, and returns a set of edges $T' \subset MST(G)$ such that the amount of vertices in the contracted graph G/T' is $O(\frac{m}{k})$.*

At the beginning, none of the vertices are marked. Each vertex has a Fibonacci heap of its neighbors. As long as there is an unmarked vertex v in the graph, we start running Prim's algorithm from it such that every added vertex is marked (along with its heap), until one of the following 2 cases happen:

1. *The number of neighbors (the size of the Fibonacci Heap) of v is $\geq k$. This assures each iteration takes no more than $O(\log k)$.*
2. *The vertex we want to add is already marked. We will add it to the MST, contract it, and then stop.*

After the iteration ends, we mark all the vertices in the current connected component, and move on to the next unmarked vertex.

Proof. All the edges chosen belong to an MST—just as in Prim—because they are minimal in some cut of G.

In the end, every connected component T' touches at least k edges. Why? If we stopped because of case 1, then this follows directly from the stopping condition. If we stopped because of case 2, then it means that we connected to another component that had already been finished before, which means the first time we stopped for this component it stopped because of case 1.

Since every edge touches no more than 2 connected components, and each component touches at least k edges, there are no more than $\frac{2m}{k}$ components.

Running Time: Using Fibonacci heaps we get $O(m + n \log k)$ because no Fibonacci heap exceeds size k. □

Example 6.14. *Prim takes $O(m + n \log n)$ time. We run the described iteration once with $k = \log n$.*
It will take $O(m \log k) = O(m \log \log n)$ time, and we will be left with a graph such that #edges $\leq m$ and #vertices $\leq \frac{2m}{\log n}$. Therefore, Prim will take $m + \frac{2m}{\log n} \cdot \log n = O(m)$ time to run, and using Fibonacci Heaps it will take $O(m + n \log \log n)$. Since Prim still takes $O(m)$, the total is $O(2m + n \log \log n)$.

Claim 6.15. *For every k we can get an algorithm that solves MST in time $O(km + n \log^{(k)} n)$ (with the universal constant not depending on m, n, k). It also balances to $O(m\beta(m, n))$ when $k = \beta(m, n)$.*

Proof. By induction. We already saw the case for $i = 1, 2$. We assume correctness for general i and prove for $i + 1$. We do one iteration of Fredman-Tarjan with $k = \log^{(i)} n$, which will take $O(m + n \log k) = O(m + n \log^{(i+1)} n)$ time. After that, we end up with a graph that has no more than m edges and $\frac{m}{\log^{(i)} n}$ vertices. Using the induction hypothesis, we can finish with $O(im + \frac{m}{\log^{(i)} n} \cdot \log^{(i)} n)$ time, which results in a total of $O((i + 2) \cdot m + n \log^{(i+1)} n)$. □

Note 6.16. *The best known deterministic algorithm for MST runs in $O(m\alpha(m, n))$ (Chazelle [6]). Using randomization, we know how to achieve linear expected running time [13].*

6.3 Finding Minimum Cut in Graphs

Given an undirected and non-weighted graph G, the goal is to find $\emptyset \subsetneq S \subsetneq V$ such that $e(S, S^c)$ is minimal.

In the Algorithms course, we saw a solution using MaxFlow with runtime $O(mn^2)$. We will want to find simple algorithms for MinCut that use contractions and randomization (they will have a probability of $o(1)$ of outputting a non-minimal cut).

Algorithm 6.17 (Karger (1993))**.** *We will see an algorithm that has a running time of $\tilde{O}(m)$, and returns a minimal cut of the graph with probability $\geq p$ for some very small p.*

We will run the algorithm $\frac{100}{p}$ times and take the minimal cut from all the outputs.

The algorithm: As long as there are more than 2 vertices in the graph, we take a uniformly random edge e, contract it (allowing parallel edges but no self-loops), and continue the same procedure with G/e. When there are only 2 vertices left in the graph, we return the cut that those vertices define.

The intuition for the algorithm: most of the edges do not cross the minimal cut, so by taking an arbitrary edge, we will likely get an edge that does not cross the cut.

Claim 6.18. *Let (S, S^c) be a minimum cut in a graph G. The size of the cut is $\leq \frac{2m}{n}$.*

Proof. The average degree of the graph is $\frac{2m}{n}$. Therefore, there exists a vertex u with degree $\leq \frac{2m}{n}$. Since the cut $(\{u\}, V \setminus \{u\})$ already defines a cut of G with size $\leq \frac{2m}{n}$, the size of the minimum cut must be $\leq \frac{2m}{n}$. □

Corollary 6.19. *Let (S, S^c) be a minimum cut in a graph G. The probability that a uniformly randomly picked edge crosses it is $\leq \frac{2m/n}{m} = \frac{2}{n}$.*

Claim 6.20. *The probability of Karger's algorithm to output the minimum cut (S, S^c) is $\geq \frac{1}{\binom{n}{2}}$.*

Proof. There are $n-2$ iterations, each on $n, n-1, n-2, \ldots, 3$ vertices. When there are t vertices, the probability of not picking an edge that crosses the cut is $\geq 1 - \frac{2}{t}$. Through all the iterations we get a probability:

$$\geq \left(1 - \frac{2}{n}\right)\left(1 - \frac{2}{n-1}\right)\left(1 - \frac{2}{n-2}\right)\cdot\ldots\cdot\left(1 - \frac{2}{3}\right) = \frac{n-2}{n}\cdot\frac{n-3}{n-1}\cdot\frac{n-4}{n-2}\cdot\ldots\cdot\frac{1}{3} = \frac{2\cdot 1}{n(n-1)} = \frac{1}{\binom{n}{2}}.$$

□

Note 6.21. *Every minimal cut in a contracted graph is also a cut in the original graph. Therefore, it will not be less than the minimum cut.*

We get a simple algorithm that has a run time of $\tilde{O}(m \cdot n^2)$, since each iteration takes $\tilde{O}(m)$ time, and there are no more than $n^2 \geq \binom{n}{2}$ iterations. Each iteration is "Linear" or $m \cdot \alpha(m, n)$ because it is equivalent to running Kruskal with an arbitrary order of the edges.

Corollary 6.22. *In every undirected, non-weighted graph G, there are at most $\binom{n}{2}$ minimum cuts.*

Proof. Each minimum cut has a probability of $\frac{1}{\binom{n}{2}}$ being returned as the output of Karger's algorithm, and these probabilities cannot be summed to more than 1. □

6.3.1 Improved Algorithm of Karger-Stein

Algorithm 6.23.

- *Take a random edge and contract it as before, until we are left with $\lfloor \frac{n}{\sqrt{2}} \rfloor$ contracted vertices/connected components.*
- *Run on the remaining graph the same algorithm twice recursively, and output the minimum cut between the runs.*

Note 6.24. *If at the beginning we have $n \leq 5$, we will solve it manually.*

Observation 6.25. *The algorithm without the recursive parts takes a running time of $\tilde{O}(m)$.*

Observation 6.26. *The probability that the contraction steps didn't ruin the minimum cut is:*

$$\frac{n-2}{n}\cdot\frac{n-3}{n-1}\cdot\ldots\cdot\frac{n-(n-\lceil\frac{n}{\sqrt{2}}\rceil)}{n-(n-\lceil\frac{n}{\sqrt{2}}\rceil)+2} = \frac{(\lceil\frac{n}{\sqrt{2}}\rceil+1)(\lceil\frac{n}{\sqrt{2}}\rceil)}{n(n-1)} \geq \frac{\frac{n^2}{2}}{n^2} = \frac{1}{2}.$$

Run time: Before the recursion, the algorithm takes $\tilde{O}(m)$, and then there are 2 recursive calls on graphs of $\frac{n}{\sqrt{2}}$ vertices. We will need to beware of parallel edges, but it is not really a problem (can be seen by analyzing directly or by replacing parallel edges with a weighted edge and have the distribution of the randomization according to the weights). Therefore, the run time of the "iteration" stage is $\tilde{O}(n^2)$. The depth of the recursion is $\log_{\sqrt{2}} n = \Theta(\log n)$. The recursion time is:

$$T(n) = cn^2 \log n + 2T\left(\frac{n}{\sqrt{2}}\right) = cn^2 \log n + 2 \cdot c \cdot \left(\frac{n}{\sqrt{2}}\right)^2 \log\left(\frac{n}{\sqrt{2}}\right) + 4T\left(\frac{n}{2}\right) \leq \ldots \leq c' n^2 \log^2 n = \tilde{O}(n^2)$$

What is the probability of success for the algorithm? We can analyze it as a complete binary tree of height $\log_{\sqrt{2}} n$, such that each node is erased independently with a probability of $\frac{1}{2}$, and ask what is the probability that a full path from the root to some leaf survives.

Note 6.27. *In statistics, this is called the Galton–Watson branching process for complete k-ary trees of height h and removal probability p for each vertex.*

Exercise 6.28. *Denote by p_h the probability that at least one root-to-leaf path "survived" the removals.*

(a) If $p > \frac{1}{k}$, prove that there exists a constant $\alpha > 0$ such that $p_h \geq \alpha$ for all $h \in \mathbb{N}$.

(b) If $p = \frac{1}{k}$, prove that $p_h = \Omega\left(\frac{1}{h}\right)$.

(c) If $p < \frac{1}{k}$, prove that there exists a constant $\beta \in (0,1)$ such that $p_h = O(\beta^h)$.

Proof of Karger-Stein Success Probability. We would like to show that our probability of success is $\geq \frac{1}{h+1}$ where h is the height of the tree. By induction: we denote the probability of success when doing i (here $h = i$) iterations by p_i.

- **Base:** $p_0 = 1$
- $p_1 = \frac{1}{2}$
- $p_2 = \frac{1}{2} \cdot \left(1 - \left(\frac{1}{2}\right)^2\right) = \frac{1}{2} \cdot \frac{3}{4} = \frac{3}{8} \geq \frac{1}{3}$.
- **Step:** $p_{i+1} = \frac{1}{2} \cdot (1 - (1 - p_i)^2) = \frac{1}{2}(1 - 1 + 2p_i - p_i^2) = \frac{1}{2} p_i (2 - p_i)$

In the range $[0, 2]$ this function is increasing, and using the induction hypothesis that $p_i \geq \frac{1}{i+1}$, it gives us:

$$\frac{1}{2} p_i (2 - p_i) \geq \frac{1}{2} \cdot \frac{1}{i+1}\left(2 - \frac{1}{i+1}\right) \geq \frac{1}{2} \cdot \frac{1}{i+1}\left(2 - \frac{1}{i}\right) = \frac{1}{2(i+1)} \cdot \frac{2i-1}{i} \geq \frac{1}{i+2}.$$

□

From the analysis, we get an algorithm that returns a correct result with probability $1 - o(1)$ and running time of $\tilde{O}(n^2)$, which is tight up to $\log n$.

Note 6.29. *Today we know how to do this in $m^{1+o(1)}$ in any graph.*

6.4 Minors of graphs

Definition 6.30. *A graph H is called a minor of G, if it can result from G in a sequence of vertex removals, edge removals, and edge contractions.*

Theorem 6.31. *A graph G is Planar (which means it can be drawn on a plane in such a way that no edges cross each other) $\iff$ the graphs K_5 and $K_{3,3}$ are not minors of G.*

Theorem 6.32. *Every family of graphs that is closed under taking minors can be defined by a finite set of forbidden minors [18].*

Chapter 7

The Inclusion–Exclusion Principle in Algorithms

Date: June 15, 2026
Scribe: Ron Vulkan

7.1 Introduction

Today's lecture is about the applications of the inclusion–exclusion principle in algorithms. In particular, we will see some NP-complete problems, and show how to improve the best known algorithms using this basic yet powerful principle.

7.2 Finding a Hamiltonian Path

A Hamiltonian path is a path that visits each vertex of the graph exactly once. It is known to be NP-complete.

Algorithm 7.1 (Naive solution)**.** *Iterate over every permutation of the vertices and check whether it is a valid path — this takes $O^*(n!)$ time.*[1]

We already solved a similar problem when we dealt with color-coding: finding a path that visits every color exactly once. We solved this in $O^*(2^k)$ time, so in our case, where $k = n$, we obtain a solution in $O^*(2^n)$ time. For completeness, we will describe the solution again.

Algorithm 7.2. *We solve the problem with dynamic programming. For each $v \in V$ and $V' \subseteq V$, let $dp(v, V')$ be true if and only if there is a path that visits exactly the vertices of V' (visiting each of them exactly once) and ends at v. The base case is $dp(v, \{v\}) = true$ for every $v \in V$, and the transition is*

$$dp(v, V') = \bigvee_{\substack{u \in V' \setminus \{v\} \\ (u,v) \in E}} dp\left(u, V' \setminus \{v\}\right).$$

Thus, we can compute the whole table in $O^*(2^n)$ time.

[1] The symbol $O^*(T)$ denotes $O(T \cdot n^c)$ for some constant c, meaning up to a polynomial factor.

Issue 7.3. *We used $O^*(2^n)$ space. We would like to use less space. Can we solve this in $O^*(2^n)$ time but with polynomial space?*

Reminder 7.4 (Inclusion–Exclusion Principle)**.** *Let $A_1, A_2, \ldots, A_k \subseteq B$. Then the principle states the following.*

Union:

$$\left|\bigcup_{i=1}^{k} A_i\right| = \sum_{\emptyset \neq I \subseteq [k]} (-1)^{|I|+1} \left|\bigcap_{i \in I} A_i\right|$$

Intersection:

$$\left|\bigcap_{i=1}^{k} A_i\right| = \left|B \setminus \bigcup_{i=1}^{k} A_i^c\right| = |B| - \left|\bigcup_{i=1}^{k} A_i^c\right| =$$

$$|B| - \sum_{\emptyset \neq I \subseteq [k]} (-1)^{|I|+1} \left|\bigcap_{i \in I} A_i^c\right| = \sum_{I \subseteq [k]} (-1)^{|I|} \left|\bigcap_{i \in I} A_i^c\right| = \sum_{I \subseteq [k]} (-1)^{|I|} \left|B \setminus \bigcup_{i \in I} A_i\right|$$

Proof of the union case; the intersection case is similar. Let $x \in B$ and consider a set $I \subseteq [k]$. What will the coefficient of x be in each of the following cases?

- 0? When $x \notin \bigcap_{i \in I} A_i$, meaning there exists $i \in I$ such that $x \notin A_i$.
- $+1$? When $x \in \bigcap_{i \in I} A_i$ and $|I|$ is odd.
- -1? When $x \in \bigcap_{i \in I} A_i$ and $|I|$ is even.

Let J be the set of indices of the sets A_i that contain x. This means we get a coefficient of 0 if $I \not\subseteq J$, and $(-1)^{|I|+1}$ otherwise.

Lemma 7.5. *Let J be a non-empty set. Then the number of subsets of J of even size is equal to the number of subsets of odd size.*

Proof. Take some $i \in J$. We create a pairing of 2^J in which every subset $I \subseteq J \setminus \{i\}$ is paired with the subset $I \cup \{i\}$. This is a valid pairing, and each pair contains one even and one odd subset. □

To conclude the proof, in the case where $x \in \bigcup A_i \implies J \neq \emptyset$, we get that the sum over all I is

$$\sum_{\emptyset \neq I \subseteq J} (-1)^{|I|+1} = 0 - (-1)^{|\emptyset|+1} = 1.$$

In the case where $x \notin \bigcup A_i \implies J = \emptyset$, we get that the sum over all I is

$$\sum_{\emptyset \neq I \subseteq J} (-1)^{|I|+1} = 0.$$

□

Reminder 7.6. *We know how to compute the number of paths (not necessarily simple) of length k between every pair of vertices in polynomial time.*

Observation 7.7. *The number of Hamiltonian paths is equal to the number of length n paths that visit every vertex.*

Claim 7.8. *The number of Hamiltonian paths in a graph can be computed in $O^*(2^n)$ time and polynomial space.*

Proof. Let B denote the set of all paths of length n in the graph, and let A_v denote the set of paths of length n that visit v. By the observation, the number of Hamiltonian paths is exactly $\left|\bigcap_{v \in V} A_v\right|$. From the inclusion–exclusion principle:

$$\left|\bigcap_{v \in V} A_v\right| = \sum_{V' \subseteq V} (-1)^{|V'|} \left|\bigcap_{v \in V'} A_v^c\right|$$

Let us look at $\bigcap_{v \in V'} A_v^c$. It is the set of length n paths that do not visit any vertex of V'. In other words, it is the set of length n paths in the graph $G[V \setminus V']$.

Thus, by computing M^{n-1} (where M is the adjacency matrix of this subgraph) and summing the entries in its upper triangular half (not including the diagonal entries), we obtain the desired result. This matrix computation can be performed in polynomial time and space.

Ultimately, finding the number of Hamiltonian paths requires computing a sum over 2^n subsets. Because the required matrix operations for each subset take polynomial time and space, the entire problem can be solved in $O^*(2^n)$ time with polynomial space. □

7.3 Set Partitioning via Inclusion–Exclusion

This section follows Björklund, Husfeldt, and Koivisto [3].

Problem 7.9 (Set Partition/Cover Problems)**.** *Given a set V (of size n) and a family $\mathcal{F}$ of subsets of V, there are a few interesting questions:*

- *Can we partition V into k subsets from $\mathcal{F}$?*
- *Can we cover V with k subsets from $\mathcal{F}$?*
- *How many [covers/partitions] of V with k subsets from $\mathcal{F}$ are there?*
- *Weighted case (every subset from $\mathcal{F}$ has a weight): what is the [sum of all/max value] of the [covers/partitions] of V with k subsets from $\mathcal{F}$?*

Observation 7.10. *A graph G is k-colorable $\iff$ V can be covered by k sets from the family of independent sets of G.*

Proof.

$\Rightarrow$ Assume G has a valid k-coloring. By definition, no two adjacent vertices share the same color. Therefore, all the nodes assigned the same color form an independent set. Since every vertex in V receives exactly one of the k colors, these k independent sets completely cover (and partition) V.

$\Leftarrow$ Assume V is covered by k independent sets $I_1, I_2, \ldots, I_k$. We color each node based on the index of an arbitrary set it appears in. Because every two nodes that share a color come from the same independent set, there are no edges between them. Thus, this constitutes a valid k-coloring of the graph.

□

How fast can we check whether G is k-colorable?

Algorithm 7.11 (Naive)**.** *Try every possible coloring, in $O^*(k^n)$ time.*

Algorithm 7.12. *Using dynamic programming, we find for each $V' \subseteq V$ the minimal number of independent sets needed to cover it (therefore, this number for V will be exactly $\chi(G)$). The table will contain exactly 2^n entries, one for each $V' \subseteq V$. We will iterate over them in increasing order of size.*

Base: $T[\emptyset] = 0$.

Step: *In order to compute $T[V']$, we will iterate over all independent sets U such that $\emptyset \neq U \subseteq V'$ (we check every subset, and if it is an independent set we proceed). We compute $1+T[V' \setminus U]$ and take the minimum of these values across all checked subsets U.*

Running time: We iterate over every subset of every subset,

$$\sum_{V' \subseteq V} 2^{|V'|} = \sum_{i=0}^{n} \binom{n}{i} \cdot 2^i \cdot 1^{n-i} = (2+1)^n = 3^n,$$

therefore the algorithm runs in $O^*(3^n)$ time.

Our goal is to solve all the set-partition problems in $O^*(2^n)$ time (for a general $\mathcal{F}$ this is the best we can hope for, as it is the size of the input). We can think of $\mathcal{F}$ as a function $f : 2^V \to \{0, 1\}$.

Definition 7.13 (Zeta Transform)**.** *Given a function $f : 2^V \to \{0, 1\}$, define $\hat{f} : 2^V \to \mathbb{N}$ as follows:*

$$\hat{f}(U) = \sum_{U' \subseteq U} f(U')$$

This transform is due to Rota [19]. How do we compute it?

Claim 7.14. *$\hat{f}$ can be computed in $O^*(2^n)$ time.*

Algorithm 7.15 (Yates [24])**.** *We maintain a series of functions $f = f_0, f_1, \ldots, f_{n-1}, f_n = \hat{f}$, and arrange V in an arbitrary order $\{1, 2, \ldots, n\}$. For every $V' \subseteq V$:*

$$f_{i+1}(V') = \begin{cases} f_i(V') + f_i(V' \setminus \{i+1\}) & \text{if } (i+1) \in V' \\ f_i(V') & \text{if } (i+1) \notin V' \end{cases}$$

Claim 7.16. *By induction over i, the value $f_i(V')$ sums f over all the subsets of V' whose elements after index i match those in V'.*

This means that indeed $f_n = \hat{f}$. Overall, the algorithm for computing $\hat{f}$ runs in $O(n2^n) = O^*(2^n)$ time (and space).

Theorem 7.17. *We can compute the number of k-covers of V in $O^*(2^n)$ time.*

Proof. Let B denote the set of all k-tuples of sets from $\mathcal{F}$. A cover of size k is exactly a k-tuple in which every element of V is present in at least one of the subsets. Let $A_v \subseteq B$ denote all the k-tuples in which v appears in at least one of the sets.

Thus, the problem is equal to computing $\left|\bigcap_{v\in V} A_v\right|$. From the inclusion–exclusion principle:

$$\left|\bigcap_{v\in V} A_v\right| = \sum_{V'\subseteq V} (-1)^{|V'|} \cdot \left|\bigcap_{v\in V'} A_v^c\right|$$

But what is $\bigcap_{v\in V'} A_v^c$? It is the set of all k-tuples which do not contain any element from V'. In other words, every subset in the k-tuple does not contain any element from V'.

We claim that $\left|\bigcap_{v\in V'} A_v^c\right| = \hat{f}(V \setminus V')^k$. This is true because every element in the k-tuple can be any element of $\mathcal{F}$ which is also a subset of $V \setminus V'$, and the number of such elements is exactly $\hat{f}(V \setminus V')$.

Thus we finish our proof, as we can compute $\hat{f}$ in $O^*(2^n)$ time, and then compute the final sum in $O^*(2^n)$ time as well. □

Note 7.18. *We only solved covers, not partitions. Define $f_i(U) = f(U) \cdot \mathbb{1}_{|U|=i}$. Notice that $\hat{f}_i(U)$ denotes the number of subsets of U that belong to $\mathcal{F}$ and have size exactly i.*

Observation 7.19. *A cover is also a partition $\iff$ the sum of the subset sizes is n.*

Conclusion 7.20. *Define B and A_v as such k-tuples like before, but with sum of sizes equal to n. Then the number of partitions is:*

$$\begin{aligned}\left|\bigcap_{v\in V} A_v\right| &= \sum_{V'\subseteq V} (-1)^{|V'|} \cdot \left|\bigcap_{v\in V'} A_v^c\right| \\ &= \sum_{V'\subseteq V} (-1)^{|V'|} \sum_{i_1+i_2+\cdots+i_k=n} \prod_{j=1}^{k} \hat{f}_{i_j}(V \setminus V')\end{aligned}$$

This is because $\bigcap_{v\in V'} A_v^c$ now requires us to choose a k-tuple whose subset sizes sum to n. Thus, it is sufficient to compute $\sum_{i_1+i_2+\cdots+i_k=n} \prod_{j=1}^{k} \hat{f}_{i_j}(V \setminus V')$ for every V'. We will compute this value using dynamic programming. Define the solution with parameters n', k' as $dp(n', k')$. Thus:

$$\begin{aligned}dp(n,k) &= \sum_{i_1+i_2+\cdots+i_k=n} \prod_{j=1}^{k} \hat{f}_{i_j}(V \setminus V') \\ &= \sum_{i_k=0}^{n} \hat{f}_{i_k}(V \setminus V') \cdot \left[\sum_{i_1+i_2+\cdots+i_{k-1}=n-i_k} \prod_{j=1}^{k-1} \hat{f}_{i_j}(V \setminus V')\right] \\ &= \sum_{i_k=0}^{n} \hat{f}_{i_k}(V \setminus V') \cdot dp(n-i_k, k-1)\end{aligned}$$

Therefore, given $\hat{f}$ we can compute the entire DP table in $O(n^2k)$ time, and $dp(n,k)$ is the value we needed. Ultimately, we have shown that the number of partitions of size k can be computed in $O^*(2^n)$ time.

Exercise 7.21.

- *Show how to count the number of [covers/partitions] if instead of $\mathcal{F}$ there are k families of sets $\mathcal{F}_1, \mathcal{F}_2, \ldots, \mathcal{F}_k$, and a [cover/partition] takes one set from every family.*

- *Conclude that it is possible to solve the List-Coloring problem in $O^*(2^n)$ time.*
- *Show how to sum the weight of all [covers/partitions]. Meaning that instead of f, we have w which is a general function (not only into $\{0,1\}$), and the weight of a [cover/partition] is $\prod_{i=0}^{k} w(V_i)$.*
- *Show how to find a legal k-coloring in $O^*(2^n)$ time.*

Claim 7.22 (Reduction)**.** *An algorithm which computes the sum of weights of [covers/partitions] $\rightarrow$ an algorithm which computes the maximum weight of a [cover/partition] (of a somewhat limited function w).*

Proof. Assume that w takes only integer values. Define M as the maximum absolute value w takes over 2^V. We will take a very large β, and define a new function $w'(u) = \beta^{w(u)}$. We run the sum-algorithm on w', and we obtain:

$$\sum_{V_1,\ldots,V_k \text{ covers}} \prod_{i=1}^{k} w'(V_i) = \sum_{V_1,\ldots,V_k} \beta^{\sum_{i=1}^{k} w(V_i)}$$

We take $\beta > (2^n)^k$ (which is the maximum number of [covers/partitions]). Since β is larger than the total possible number of covers/partitions, interpreting this sum in base β allows us to extract the exact number of covers/partitions for any given weight, thereby easily finding the maximum [cover/partition] weight.

Note that in w' the maximum weight is $\beta^M = 2^{nkM}$, which means that this reduction works only for a polynomial M (in order to keep the representation length of the numbers polynomial). □

Note 7.23. *All the algorithms we presented work with very large numbers ($\approx 2^{nk}$). This is fine because they can be represented using $O(nk)$ bits. Therefore, basic arithmetic operations take polynomial time in the length of the representation, yielding an $O^*(1)$ overhead.*

Exercise 7.24. *Show how to find a max k-cut in $O^*(2^n)$ time.*

Open problem: Is it possible to compute $\chi(G)$ in $O^*(2^n)$ time and polynomial space?

Chapter 8

Expanders

Date: June 22, 2026
Scribe: Nicolas Duek

8.1 Introduction

8.1.1 What is an Expander?

We want a graph that is, on the one hand, "average" or highly connected, and on the other hand, sparse.

In computer science applications, we often require the graph to be "explicit":

1. There is a fast algorithm that produces the graph.
2. It might not be strictly necessary to store the graph in memory. Instead, there is a fast algorithm that, given a node's index, outputs the indices of its neighbors.

8.2 Definitions

Definition 8.1 (Mixing property)**.** *For every pair of subsets $A, B \subseteq V$, the number of edges between them satisfies*

$$|E(A,B)| \approx |A||B|\frac{m}{\binom{n}{2}}$$

Definition 8.2 (Edge expansion)**.** *For every $S \subseteq V$ such that $|S| \leq \frac{|V|}{2}$,*

$$|E(S, S^C)| \geq \varphi|S|$$

where φ is the expansion (constant) parameter.

Definition 8.3 (Vertex expansion)**.** *For every $S \subseteq V$ such that $|S| \leq \frac{|V|}{2}$,*

$$|N(S)| \geq \varphi|S|$$

Definition 8.4 (Volume and conductance)**.** *We define for each subset $S \subseteq V$:*

$$\mathrm{Vol}_G(S) := \sum_{v \in S} \deg_G(v)$$

The conductance of a graph G is defined to be:

$$\varphi(G) := \min_{\emptyset \neq S \subsetneq V} \frac{|E(S, S^C)|}{\min(\mathrm{Vol}_G(S), \mathrm{Vol}_G(S^C))}$$

Note: If the graph is d-regular, then $\mathrm{Vol}_G(S) = d|S|$.

8.2.1 Examples

- **Path graph (P_n)?** No, because there are very sparse cuts.
- **Cycle graph (C_n)?** No, for the same reason.
- **Star graph ($K_{1,n-1}$)?** Depends. It is an edge expander, but not necessarily a vertex expander.
- **Clique (K_n)?** Very expanding.

$$\min_{1 \leq k \leq \frac{n}{2}} \frac{k(n-k)}{(n-1)k} \approx \frac{1}{2}$$

- **Random graph?** Yes. (We will see the computation later; intuitively, being an expander is equivalent to having no "surprisingly" sparse cuts).

8.2.2 Why is it Useful?

- We can construct a network that is "collapse resistant".
- **Inherent problem:** If the degree is d, then surely we can disconnect some node with d edge erasures (by erasing all edges connected to it).

Claim 8.5. *Let G be a φ-expander and let $F \subseteq E$ be a subset of its edges. Denote by $C_1, \ldots, C_r$ all connected components of $G \setminus F$ with $\mathrm{Vol}_G(C_i) \leq \frac{1}{2}\mathrm{Vol}_G(V)$. Then*

$$\sum_{i=1}^{r} \mathrm{Vol}_G(C_i) \leq O\left(\frac{|F|}{\varphi}\right)$$

Here the hidden constant is universal. Note also that there is at most one "big" connected component, namely one whose volume is strictly greater than $\frac{1}{2}\mathrm{Vol}_G(V)$.

Proof. Look at the small connected components $C_1, \ldots, C_r$ and the big component C_0 of $G \setminus F$.

$$\sum_{i=1}^{r} \varphi \,\mathrm{Vol}_G(C_i) \leq \sum_{i=1}^{r} |E(C_i, C_i^C)| \leq 2|F|$$

The first inequality follows from the definition of an expander (with C_i being the smaller side of the cut). The second inequality follows from the fact that all edges of G between the components of $G \setminus F$ belong to F, and each of them is counted at most once from each side. □

Claim 8.6. *If G is a φ-expander and $F \subseteq E$, $u, v \in V$, it can be checked in $O\left(\frac{|F|}{\varphi}\right)$ time whether u and v are connected in $G \setminus F$.*

Proof. We start a BFS from each of the nodes u and v and stop if we have seen $\frac{2|F|}{\varphi}$ edges. If one of the searches finds its entire connected component, then it knows whether the other node is in it. If neither manages to finish, then they both must belong to the big connected component C_0. In particular, this implies there is a path between them. □

Claim 8.7. *If G is a φ-expander, then* $\operatorname{diam}(G) = O\left(\frac{\log n}{\varphi}\right)$. *In expanders, everything is close to each other.*

Note: Can we do better than $O(\log n)$? No, because if the maximal degree Δ is constant, then at distance at most k from a given node there are at most $\Delta^0 + \Delta^1 + \ldots + \Delta^k$ vertices.

Proof. Let v be any node. Define a sequence of sets:

$$\{v\} = S_0 \subseteq S_1 \subseteq S_2 \subseteq \ldots \subseteq S_k$$

such that $S_{i+1} := S_i \cup N(S_i)$.

If $\operatorname{Vol}_G(S_i) \leq \frac{1}{2}\operatorname{Vol}_G(V)$, then from the expander definition, $\frac{|E(S_i, S_i^C)|}{\operatorname{Vol}_G(S_i)} \geq \varphi$. Therefore,

$$\operatorname{Vol}_G(S_{i+1}) \geq \operatorname{Vol}_G(S_i) + |E(S_i, S_i^C)| \geq (1+\varphi)\operatorname{Vol}_G(S_i)$$

Denote by k the first index for which $\operatorname{Vol}_G(S_k) \geq \frac{1}{2}\operatorname{Vol}_G(V)$. Since $\log(1+\varphi) \geq \frac{\varphi}{2}$ for every $\varphi \in (0, 1]$, the following holds:

$$k \leq \log_{(1+\varphi)}\left(\frac{1}{2}\operatorname{Vol}_G(V)\right) = \frac{\log\left(\frac{1}{2}\operatorname{Vol}_G(V)\right)}{\log(1+\varphi)} \leq \frac{2\log\left(\frac{1}{2}n^2\right)}{\varphi} = O\left(\frac{\log n}{\varphi}\right)$$

Therefore, for all $u, v \in V$, if we look at the sets of vertices at a distance $O\left(\frac{\log n}{\varphi}\right)$ from each, each such set has a volume strictly greater than $\frac{1}{2}\operatorname{Vol}_G(V)$. By the pigeonhole principle, they must share an edge, bounding the diameter. □

Definition 8.8. *A bipartite graph $(V = L \uplus R, E)$ is called an $(n, m, d, \gamma, \varphi)$-expander if:*

1. *$|L| = n$ and $|R| = m$.*
2. *The degree of all nodes in L is exactly d.*
3. *$\forall S \subseteq L$ of size $|S| \leq \gamma n$, we have $|N(S)| \geq \varphi|S|$.*

Note: Usually we will assume $n \geq m$, but $m = \Theta(n)$. Ideally, $\varphi \approx d$.

Theorem 8.9. *For any constant α such that $\alpha n \leq m \leq n$, and for any $\varepsilon > 0$, there exist constants $d \in \mathbb{N}$ and $\gamma > 0$ (depending solely on α and ε) such that an $(n, m, d, \gamma, (1-\varepsilon)d)$-expander exists.*

Proof. We use the probabilistic method. Construct a random bipartite graph by having each of the n vertices in L choose d neighbors in R uniformly at random (with replacement, which only worsens expansion). We want to bound the probability that there exists a subset $S \subseteq L$ of size $k \leq \gamma n$ that fails the expansion condition $|N(S)| \geq (1-\varepsilon)d|S|$.

Fix a set $S \subseteq L$ of size k. The total number of edges emanating from S is dk. For S to fail to expand, these dk edges must land in a subset $T \subseteq R$ of size strictly less than $(1-\varepsilon)dk$.

Let us bound the probability that all dk edges land in a specific set T of size $(1-\varepsilon)dk$. The probability for a single edge is $\frac{|T|}{m} = \frac{(1-\varepsilon)dk}{m}$. Since the edges are chosen independently, the probability that all dk edges land in T is:

$$\left(\frac{(1-\varepsilon)dk}{m}\right)^{dk}$$

We now apply the union bound over all choices of $S \subseteq L$ of size k, and all choices of $T \subseteq R$ of size $(1-\varepsilon)dk$. The probability P_k that *any* set of size k fails is at most:

$$P_k \leq \binom{n}{k}\binom{m}{(1-\varepsilon)dk}\left(\frac{(1-\varepsilon)dk}{m}\right)^{dk}$$

Using the standard inequality $\binom{a}{b} \leq \left(\frac{ea}{b}\right)^b$, we can bound the binomial coefficients:

$$\begin{aligned} P_k &\leq \left(\frac{en}{k}\right)^k \left(\frac{em}{(1-\varepsilon)dk}\right)^{(1-\varepsilon)dk} \left(\frac{(1-\varepsilon)dk}{m}\right)^{dk} \\ &= \left(\frac{en}{k}\right)^k e^{(1-\varepsilon)dk} \left(\frac{(1-\varepsilon)dk}{m}\right)^{\varepsilon dk} \\ &= \left[\frac{en}{k} e^{(1-\varepsilon)d} \left(\frac{(1-\varepsilon)dk}{m}\right)^{\varepsilon d}\right]^k \end{aligned}$$

Since $m \geq \alpha n$, we have $\frac{n}{k} = \frac{n}{m}\frac{m}{k} \leq \frac{1}{\alpha}\frac{m}{k}$. Substituting this into the base of the exponent, and separating the factor $((1-\varepsilon)d)^{\varepsilon d}$ from the powers of $\frac{k}{m}$, gives:

$$P_k \leq \left[\frac{e}{\alpha} e^{(1-\varepsilon)d} \left((1-\varepsilon)d\right)^{\varepsilon d} \left(\frac{k}{m}\right)^{\varepsilon d - 1}\right]^k$$

We require $d > \frac{1}{\varepsilon}$ so that the exponent $\varepsilon d - 1$ is strictly positive. Let $d = \lceil \frac{2}{\varepsilon} \rceil$. Because the exponent on (k/m) is strictly positive, the base becomes arbitrarily small as k/m approaches 0.

Since $k \leq \gamma n \leq \frac{\gamma}{\alpha} m$, we have $\frac{k}{m} \leq \frac{\gamma}{\alpha}$. Thus:

$$P_k \leq \left[\frac{e}{\alpha} e^{(1-\varepsilon)d} \left((1-\varepsilon)d\right)^{\varepsilon d} \left(\frac{\gamma}{\alpha}\right)^{\varepsilon d - 1}\right]^k$$

By choosing $\gamma > 0$ sufficiently small (which depends only on α and ε, as d is already fixed by ε), we can ensure that the term inside the brackets is at most $\frac{1}{3}$. Therefore, $P_k \leq \left(\frac{1}{3}\right)^k$.

Finally, summing over all possible sizes k from 1 to γn, the total probability that the graph fails to be an expander is strictly bounded:

$$\sum_{k=1}^{\gamma n} P_k \leq \sum_{k=1}^{\infty} \left(\frac{1}{3}\right)^k = \frac{1}{2} < 1$$

Since the failure probability is strictly less than 1, an expander with these parameters must exist. □

8.3 Error Correction Codes

- An error correction code can be viewed as a function mapping vectors of length n to vectors of length $\frac{n}{R}$ (where $R < 1$ is the rate, so the output is longer), such that the original input can be recovered even if we corrupt an ε fraction of the output.
- Alternatively, it is a set of 2^{Rn} binary vectors of length n with a minimal distance $\geq \delta n$ between each pair.
- A "good" code is one where δ and R are strictly positive constants. Such codes exist (e.g., random codes, random linear subspaces, etc.). However, decoding them is generally a hard problem.

8.3.1 Tanner Codes

Tanner codes [21] achieve constant rate R and constant δ along with linear-time decoding.

Construction: Take a bipartite expander with $\varepsilon < \frac{1}{4}$ and $m = \frac{n}{2}$. We think of the code as a linear subspace of $\mathbb{F}_2^n$ defined as follows: we think of the nodes in L as variables $x_1, \ldots, x_n$, and each node $y \in R$ represents a linear constraint:

$$\bigoplus_{(x_i, y) \in E} x_i = 0$$

(The rows of the Parity Check Matrix, or PCM, are exactly these constraints.)

Rate: The code is the linear subspace of $\mathbb{F}_2^n$ consisting of all variable assignments that satisfy all m equations. Because the equations are homogeneous, the dimension of the subspace is $\geq n - m = \frac{n}{2}$ (and therefore the rate is $R \geq \frac{1}{2}$).

Distance: For a linear code, it is sufficient to bound the minimal Hamming weight of a non-zero codeword from below.

Lemma 8.10. *Every subset $S \subseteq L$ of size $|S| \leq \gamma n$ has at least $(1 - 2\varepsilon)d|S|$ neighbors in R that are connected to exactly one node in S (we will call these "unique" neighbors).*

Proof. There are at most $\varepsilon d|S|$ nodes in R with ≥ 2 edges from S. If there were more, there would be strictly fewer than $d|S| - \varepsilon d|S|$ edges going out of S that do not share a right-endpoint, which would violate the expansion property. From the expander definition, S has at least $(1-\varepsilon)d|S|$ total neighbors. By the above observation, at most $\varepsilon d|S|$ of them receive multiple edges from S. Thus, the number of unique neighbors is at least $(1-\varepsilon)d|S| - \varepsilon d|S| = (1-2\varepsilon)d|S|$. □

Corollary 8.11. *The distance of the code is at least γn (in fact, it is even larger).*

Proof. Assume towards contradiction that there is a non-zero codeword with $\leq \gamma n$ ones. We denote the set of variables corresponding to these ones as S. From the lemma above, they have at least one unique neighbor in R (since $(1-2\varepsilon)d|S| > 0$ for $\varepsilon < 1/4$). However, a unique neighbor means that the corresponding constraint sum contains exactly one 1, so it evaluates to $1 \neq 0$. This contradicts the assumption that the constraint is satisfied. □

Claim 8.12. *If a word x is at a distance $\leq (1-3\varepsilon)\gamma n$ from a valid codeword x_0, then x_0 can be recovered from x in linear time.*

Proof / Algorithm. We write the assignment x on L and check which constraints in R are violated and which are satisfied. As long as there is a node in L for which strictly more than half of its neighboring constraints are violated, we flip its value. □

Claim 8.13. *As long as $x \neq x_0$, such a node exists.*

Proof. The difference vector $x - x_0$ has a Hamming weight of at most $(1-2\varepsilon)\gamma n$. Let S be the set of indices where they differ. As shown before, S has at least $(1-2\varepsilon)d|S|$ unique neighbors. Every such unique neighbor corresponds to a violated constraint (because it receives exactly one flipped input compared to the valid x_0).

If no variable $x_i \in S$ had more than $\frac{d}{2}$ violated constraints, then the total number of violated constraints connected to S would be at most $|S|\frac{d}{2}$. But we know the number of violated constraints is at least $(1-2\varepsilon)d|S|$, and $(1-2\varepsilon)d|S| > \frac{d}{2}|S|$ (since $\varepsilon < 1/4$), which is a contradiction. □

Claim 8.14. *When the value of such an x_i is flipped, the total number of violated constraints strictly decreases.*

Proof. Every constraint whose status changes must be a neighbor of x_i. Since x_i was chosen because strictly more than half of its neighboring constraints were violated, flipping it corrects more constraints than it breaks. □

Corollary 8.15. *Within at most m steps, we will reach 0 violated constraints (recovering x_0), provided the initial word x is at a distance of at most $(1-3\varepsilon)\gamma n$ from x_0.*

Proof. To ensure the algorithm successfully terminates, we must show that the intermediate states never drift too far from x_0. Specifically, the error reduction relies on the expansion property, which only holds for sets of size at most γn. We will prove that the distance to x_0 never reaches $(1-\varepsilon)\gamma n$.

Let x be the initial word with $\text{dist}(x, x_0) \leq (1 - 3\varepsilon)\gamma n$. Since each flipped variable participates in exactly d constraints, the initial number of violated constraints is at most:

$$d \cdot \text{dist}(x, x_0) \leq d(1 - 3\varepsilon)\gamma n$$

By Claim 8.14, the algorithm strictly decreases the total number of violated constraints at every step.

Assume towards contradiction that during the process, we reach a state x' that is at a distance of exactly $(1 - \varepsilon)\gamma n$ from x_0. Let S' be the set of variables where x' differs from x_0. Since $|S'| = (1 - \varepsilon)\gamma n \leq \gamma n$, we can safely apply Lemma 8.10. The set S' must have at least $(1 - 2\varepsilon)d|S'|$ unique neighbors in R.

Because each unique neighbor represents a constraint connected to exactly one incorrect variable, every unique neighbor evaluates to $1 \neq 0$ and is therefore a violated constraint. Thus, at state x', the number of violated constraints is at least:

$$(1 - 2\varepsilon)d|S'| = (1 - 2\varepsilon)d(1 - \varepsilon)\gamma n = (1 - 3\varepsilon + 2\varepsilon^2)d\gamma n$$

Since $\varepsilon > 0$, we have $1 - 3\varepsilon + 2\varepsilon^2 > 1 - 3\varepsilon$. This implies that state x' has strictly *more* violated constraints than the initial state x. This is a direct contradiction, as the number of violated constraints must strictly decrease in every step of the algorithm.

Therefore, the distance to x_0 is strictly bounded by $(1 - \varepsilon)\gamma n < \gamma n$ throughout the entire execution. Because the distance never exceeds γn, the expansion property always holds, and we can continue finding a variable to flip until 0 constraints are violated, successfully recovering x_0. Since the number of violated constraints decreases by at least 1 per step, and starts at $\leq m$, the process terminates in at most m steps. □

Chapter 9

Explicit Expanders, Expander Decomposition, and Fast Min-Cuts

Date: June 29th, 2026
Scribe: Itai Halperin

9.1 Introduction

Reminder: We defined $\varphi(G) = \min_{\emptyset \neq S \subsetneq V} \frac{|E(S,S^c)|}{\min(\mathrm{Vol}(S),\mathrm{Vol}(S^c))}$, and showed the existence, properties, and uses of expanders.

Plan: How we build explicit expanders (without proof), some more "general" uses, and Expander Decomposition.

9.2 Explicit Expander Constructions

We can think of different levels of explicitness (from the least to the most explicit):

1. An existence proof.
2. An efficient algorithm that returns an expander w.h.p.
3. An efficient algorithm in expectation that always returns an expander (which just requires the previous algorithm + checking efficiently if a graph is an expander).
4. An efficient, deterministic algorithm.
5. An expander that can be represented implicitly, efficiently: i.e., given an index i we can find $N(i)$ efficiently (using a deterministic algorithm).

The ability to implicitly represent an expander is useful when the expander itself is very large, since we don't need to store it all but just store references to vertices, which only require $O(\log n)$ memory.

9.2.1 How do we even build expanders?

- **Random construction:** Works, *very* inexplicit.
- **Constructions from properties** (say algebraic ones) of certain objects, e.g.:
  - **Cayley Graphs**: We take a graph whose vertices are elements in a group G and the neighbors of each vertex $g \in G$ are all the products of g (say from the right) with some generator set $S \subseteq G$ (for an undirected graph we'll want an S that is closed w.r.t. the inverse).

    For example: A circle C_n is a Cayley graph in $\mathbb{Z}_n$ with $S = \{\pm 1\}$.
  - A series of results (mostly from the early 1990's: Phillips, Lubotzky, Margulis, Sarnak); for a number of groups, random sets of generators give expanders (even very good ones).

    Specific example: If we look at $\mathbb{F}_p^*$ (which is all the numbers except 0 modulo some prime), and for any element a we connect an edge to $a+1$, $a-1$, and $\frac{1}{a}$, we get an expander.

9.3 Another Use of Expanders

We'll see that given an expander, we can reduce the amount of randomization we need to "run" a random algorithm.

Settings Reminder: A random algorithm will be said to have a "one-sided error" if:

- "yes" is "yes"
- "no" sometimes with error (with probability ε)

In any such setting we can use amplification. We'll run k independent times, and accept if at least one of the runs yielded a "yes". Error probability was reduced to ε^k.

How much did it cost us? k times the runtime, k times the "random bits".

- We'll discuss situations where using more randomization is too expensive.

We'll see how a given expander helps reduce ε without requiring any more random bits.

Theorem 9.1. *Given a "good" (and fully explicit) bipartite graph which is a $(2^r, 2^r, k, \gamma, \varphi)$-expander (where r is the number of random bits the algorithm needs and $\gamma > \frac{\varepsilon}{k\varphi}$), we can reduce the error probability from ε to $\Theta\left(\frac{\varepsilon}{k}\right)$ while increasing the runtime k-fold but without using more than r random bits.*

Proof. We'll think of each side of the expander as a list of all possible r random bits (see Figure 9.1). Our algorithm selects one vertex at random from L (the left side, requiring r random bits), and then runs the original algorithm on the random bits of each of its k neighbors in R. Since this algorithm has a one-sided error, it reports "yes" iff at least one neighbor's random bits resulted in a "yes" result.

Let $B \subseteq L$ denote all the vertices in L s.t. *all* their neighbors in R have randomness bits that lead to error. Thus, the new error probability is $\frac{|B|}{2^r}$.

We know that all the neighbors of B are random bits that fail the algorithm, and therefore $|N(B)| \leq \varepsilon 2^r$. Assume for the sake of contradiction that $|B| > \frac{\varepsilon 2^r}{k\varphi}$, which implies that there exists $B' \subseteq B$ of size "just slightly" bigger than $\frac{\varepsilon 2^r}{k\varphi}$, and by the expander property the number of adjacent vertices in R is at least $\varphi \cdot |B'| \cdot k > \varepsilon \cdot 2^r$ and this is a contradiction (since by assumption the expander satisfies $\gamma > \frac{\varepsilon}{k\varphi}$).

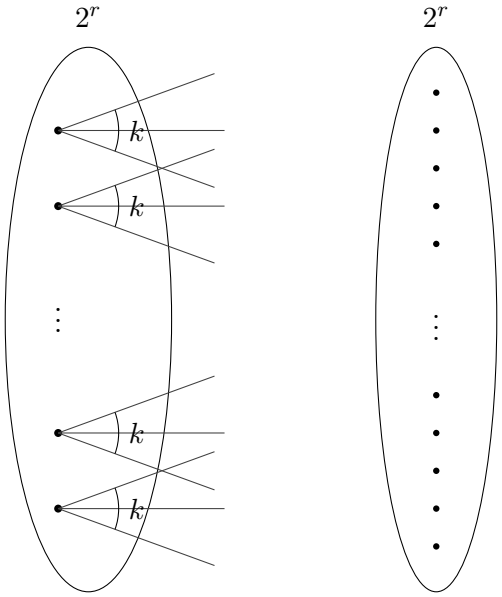

Figure 9.1: Bipartite Expander

□

Intuition: If an expander approximates a random graph, then the neighbors of a random vertex are an independently chosen random group.

We can also get exponential decay of the error for just a little more randomization (a random walk on the expander yields this).

The uses we've seen so far were:

- Constructions of things with good properties.
- Regular problems that are easier to solve on expanders (say connectivity).

Can we use expanders to solve "regular" problems on "regular", general input, faster? In recent years a paradigm of "expander decomposition" has emerged, where we show:

- It's easier to solve problems on an input that is an expander.
- Every graph can be "decomposed into expanders", and we can use an algorithm that runs efficiently on them, and somehow conclude an answer for the entire graph.

We demonstrate the method with an example: MinCut algorithms (we saw $n^{2+o(1)}$, and want $m^{1+o(1)}$).

Theorem 9.2. *If G is a graph with minimum degree $\delta(G)$ and also a φ-expander for $\varphi \geq \frac{2}{\delta(G)}$ then MinCut can be solved in $O(m)$ time and all the MinCuts are singletons.*

Proof. It's enough to show that all the MinCuts are singletons. We'll denote the size of the minimum cut by $\lambda(G) = \min_{S \subseteq V} |E(S, S^c)| \leq \delta(G)$. Let (S, S^c) be the minimum cut and assume w.l.o.g. that $\text{Vol}(S) \leq \text{Vol}(S^c)$. Then:

$$\varphi \cdot |S| \cdot \delta(G) \leq \varphi \cdot \text{Vol}(S) \leq |E(S, S^c)| \leq \delta(G) \Longrightarrow |S| \leq \frac{1}{\varphi} \leq \frac{\delta(G)}{2}$$

Assume for the sake of contradiction that $2 \leq |S| \leq \frac{\delta(G)}{2}$, so there exist two vertices $u_1 \neq u_2 \in S$. Both have at least $\underbrace{\delta(G)}_{\text{minimum degree}} - \underbrace{(|S|-1)}_{\text{maximum amount of edges inside S}}$ edges that cross (S, S^c) which is strictly bigger than $\frac{\delta(G)}{2}$ and hence:

$$|E(S, S^c)| \geq E(u_1, S^c) + E(u_2, S^c) > 2 \cdot \frac{\delta(G)}{2} = \delta(G) \geq \lambda(G)$$

which is a contradiction. Thus, $|S| = 1$. □

9.4 Expander Decomposition

We want to take a general graph G and a parameter φ, and partition the graph vertices into sets $V = V_1 \oplus V_2 \oplus \cdots \oplus V_k$ such that:

- Every $G[V_i]$ is a φ-expander.
- The number of edges between every $V_i \neq V_j$ overall is bounded by $m' \ll m$.

Theorem 9.3. *For any G, φ there exists a composition into expanders with $m' \leq 4m \cdot \varphi \cdot \log n$.*

Proof. We'll prove this using a (currently inefficient) algorithm:

If G is already a φ-expander we're done ($m' = 0, V_1 = V$).

Otherwise, there exists a cut that isn't φ-expanding, say (S, S^c). We'll add all the edges in the cut to m' and continue recursively on S, S^c individually.

We want to bound the number of edges m' the algorithm "threw away". Every time we erased edges of some cut (S, S^c) (w.l.o.g., $\text{Vol}(S) \leq \text{Vol}(S^c)$), the number of edges was $< \varphi \cdot \text{Vol}(S)$ so we "charge" (as in amortization arguments) every vertex in the "small side" (S) with the erasure of $\varphi \cdot \deg(v)$ edges. Throughout the entire run, every time a specific vertex v was charged (with $\varphi \cdot \deg(v)$), the Vol of its component is reduced by at least $\frac{1}{2}$. Therefore, the number of times a vertex can be charged is $\leq \log(\text{Vol}(G)) \leq 2 \log n$ and therefore v was charged at most $2 \log n \cdot \varphi \cdot \deg(v)$ and summing over all the vertices we get $m' \leq 4m \cdot \varphi \cdot \log n$. □

What do we need for efficiency?

- We need an algorithm that checks if a graph is an expander (we need: either guarantees a graph is a φ-expander or finds a $c\varphi$-cut) – we'll see one in the next lesson.

- How we bound the recursive runtime if (S, S^c) are very unbalanced.

We'll see how if, instead of taking any cut with *some* $\leq \varphi$ expansion, we take the most balanced one amongst them (i.e., one that maximizes $\mathrm{Vol}(S)$) then it is already sufficiently balanced.

Proof. Assume (S_1, S_1^c) is the cut with the biggest $\mathrm{Vol}(S_1)$ and $< \varphi$ expansion (and w.l.o.g., $\mathrm{Vol}(S_1) \leq \mathrm{Vol}(S_1^c)$). Let (S_2, S_2^c) be such a cut inside $G[V \setminus S_1]$ (if there isn't such a cut in $G[V \setminus S_1]$ then the recursion is done). In the original graph, $S_1 \cup S_2$ is also a cut with $< \varphi$ expansion and bigger than S_1 and so we get:

$$\underbrace{\mathrm{Vol}(S_1)}_{\leq \frac{1}{2}\mathrm{Vol}(V)} + \underbrace{\mathrm{Vol}(S_2)}_{\leq \frac{1}{2}\mathrm{Vol}(V)} > \frac{1}{2}\mathrm{Vol}(V)$$

Hence, either $\frac{1}{4}\mathrm{Vol}(V) \leq \mathrm{Vol}(S_1) \leq \frac{1}{2}\mathrm{Vol}(V)$ or $\mathrm{Vol}(S_1) < \frac{1}{4}\mathrm{Vol}(V)$ in which case:

$$\frac{1}{2}\mathrm{Vol}(V) < \mathrm{Vol}(S_1 \cup S_2) = \mathrm{Vol}(S_1) + \mathrm{Vol}(S_2) \leq \frac{3}{4}\mathrm{Vol}(V)$$

and in either case we have a sufficiently balanced cut. □

Exercise 9.4. *Assume we have an algorithm that runs in $m^{1+o(1)}$ returns either a guarantee that the graph is a φ-expander, or a cut of expansion $\leq \varphi \cdot \mathrm{polylog}(n)$ and size (max size of a φ cut)$\cdot \frac{1}{\mathrm{polylog}(n)}$. Show that we can construct a $(\varphi, m \cdot \varphi \cdot \mathrm{polylog}(n))$-decomposition into expanders in time $m^{1+o(1)}$.*

There exists such an algorithm with a $\log^2 n$ approximation for both parameters. Next week we'll see an algorithm that does the first part (checks if a graph is an expander or gives an approximate cut) but probably not one that returns a balanced cut. Now back to MinCut on a general graph.

Gabow ('95) Algorithm

MinCut can be solved in $\tilde{O}(m \cdot \lambda(G))$ [9]. (We won't see this, but we'll use it).

Idea for solving MinCut: We'll use Expander Decomposition with $\varphi = \frac{100}{\delta(G)}$. On the one hand we can hope that "inside the expanders" the problem is trivial, and on the other between the components there are only $m \cdot \frac{\mathrm{polylog}\, n}{\delta(G)} = m \cdot \varphi \cdot \mathrm{polylog}\, n$ edges and therefore Gabow's algorithm will run in $m \cdot \frac{\mathrm{polylog}\, n}{\delta(G)} \cdot \lambda(G) = \tilde{O}(m)$ time.

We hope: The minimum cut won't partition any component V_i, because then we can shrink every V_i into a vertex and solve MinCut on the resulting graph.
But that is *not* necessarily true (for example, the degrees inside a component V_i could be very small), so we need an additional step.

Algorithm:

1. We run Expander Decomposition with $\varphi = \frac{100}{\delta(G)}$.

2. **Pruning:** Iteratively, while there is still a vertex $v \in V_i$ with $\deg_{V_i}(v) < \frac{2}{5}\deg_G(v)$, we turn v into its own component.

3. **Trim:** From every component V_i' from the end of the previous step we take a subcomponent $V_i'' \subseteq V_i' \subseteq V_i$ where all the vertices satisfy $\frac{1}{2}\deg_G(v) \leq \deg_{V_i'}(v)$ (all at once) with all the rest becoming singletons (their own components).

4. We shrink each component into a single vertex and run Gabow's algorithm.

We need to show:

1. Pruning and Trim didn't increase m' too much.

2. The minimum cut doesn't partition any V_i''.

1. Pruning: We will use amortization to prove why the number of edges we "delete" (i.e., edges from $v \in V_i$ inside the component before we turn v into a singleton) is $\leq O(m')$. When we prune we delete $\geq \frac{3}{5}\deg_G(v)$ inter-component edges from v's component, and add to it just $< \frac{2}{5}\deg_G(v)$ inter-component edges.

"Invariant" (not actually an invariant, strictly speaking): The sum over all the vertices that aren't singletons of their degree going outside their component.

Every prune we lower this number by at least $(\frac{3}{5} - \frac{2}{5})\deg_G(v) = \frac{1}{5}\deg_G(v)$ edges, and each time we do so we increase the number of inter-component edges by $< \frac{2}{5}\deg_G(v)$; therefore, the number of edges we delete is $\leq 2\times$ (the number of edges in the beginning, the "invariant").

2. Trim: We clearly delete only vertices with $> \frac{1}{2}$ of their degree going outside their component (all at once). Hence, we delete $\leq \frac{1}{2}\times$ (Sum of degrees) $= O(m')$ edges.

Proving the minimum cuts don't partition any V_i'' (Proof Sketch):

- Avoid a big $V_i'' \cap S$ — using the expansion property.
- Avoid a $V_i'' \cap S$ of size bigger than a small constant (1 or 2) — using the minimum degree.
- Avoid a $V_i'' \cap S$ of size 1 — the Trim step means we can decrease the size of the cut by moving the vertex to the other side.

Chapter 10

Spectral Analysis of Expanders

Date: July 6, 2026
Scribe: Eitan Elbaum

10.1 Introduction and Recap

In the previous lectures, we explored the theory of expander graphs. We covered combinatorial definitions, probabilistic constructions, and various applications. We also discussed explicit constructions, expander decompositions, and their algorithmic utility on arbitrary graphs (even when the input graph itself is not an expander).

We began examining how to utilize expander decompositions to compute a Minimum Cut (MinCut) of a graph. While we established that the time complexity is optimal, we deferred the proof that the resulting components do not cross the MinCut. Today, we will complete this proof and then transition to the spectral analysis of expanders. This spectral approach will bridge the gap from previous lectures, enabling us to quickly verify if a graph is an expander, and if not, to efficiently find a sparse cut.

Recall that given a graph $G = (V, E)$, we partitioned it into disjoint components V_i such that the number of inter-component edges is small, and each induced subgraph $G[V_i]$ is an expander with conductance $\varphi = O\left(\frac{1}{\min_{v \in V_i} \deg(v)}\right)$. We then applied two structural refinement steps:

- **Pruning:** Iteratively remove any vertex from its component if at least $2/5$ of its incident edges leave the component. We proved that this process inflates the number of inter-component edges by at most a constant factor. We denoted the refined components as V_i'.
- **Trimming:** In a single parallel step, remove any vertex that has strictly more edges going outside its component than remaining inside. This also bounds the inter-component edge inflation by a constant. The final components were denoted V_i''.

On the remaining graph, we applied a MinCut algorithm whose running time is bounded by the minimum degree multiplied by the number of edges. Since the number of edges is roughly $O\left(\frac{m}{\delta(G)}\right)$, the overall running time is linear. We now formally verify that the global MinCut cannot strictly partition any of these final components.

Let S be the smaller side of the global MinCut.

(a) $|S \cap V_i| \leq \frac{\delta(G)}{40} N$. This follows directly from the conductance guarantee of the expander components.

(b) $|S \cap V_i'| \leq 2$. Because the pruning step ensures that most edges of any vertex remain internal, assuming $|S \cap V_i'| > 2$ would imply that strictly more than $\delta(G)$ edges cross the global cut, which contradicts the minimality of the cut.

(c) $|S \cap V_i''| = 0$ (implying $S \subseteq V_i'' \setminus R$). There is at most one node from V_i' in the same component. Even if such a node exists, the trimming step guarantees it has a strong internal pull, making it strictly optimal to place it on the other side of the cut (i.e., we threshold not exactly at half, but slightly above).

10.2 Spectral Graph Theory Fundamentals

10.2.1 Linear Algebra Reminder

Recall the spectral theorem for real symmetric matrices: any real symmetric matrix $A = A^T$ is orthogonally diagonalizable. That is, there exists a diagonal matrix D and an orthogonal matrix V (where $V^T V = I$) such that $A = VDV^T$. The diagonal entries of D are the real eigenvalues of A, and the columns of V form an orthonormal basis of eigenvectors.

For a symmetric matrix A, its associated quadratic form is the function $x \mapsto x^T Ax = \sum_{i,j} A_{i,j} x_i x_j$.

Claim 10.1. *Let the eigenvalues of an $n \times n$ symmetric matrix A be sorted as $\lambda_1 \leq \lambda_2 \leq \cdots \leq \lambda_n$. Then:*

$$\lambda_n = \max_{x \neq 0} \frac{x^T Ax}{x^T x} \quad \textit{and} \quad \lambda_1 = \min_{x \neq 0} \frac{x^T Ax}{x^T x}$$

Furthermore, the second smallest eigenvalue is given by:

$$\lambda_2 = \min_{x \neq 0, x \perp v_1} \frac{x^T Ax}{x^T x}$$

where v_1 is the eigenvector corresponding to λ_1.

Proof. Since V is an orthogonal matrix, transforming the space via $y = V^{-1}x = V^T x$ preserves the Euclidean norm, i.e., $x^T x = y^T y$. Thus:

$$\max_{x \neq 0} \frac{x^T Ax}{x^T x} = \max_{y \neq 0} \frac{y^T (V^{-1} AV) y}{y^T y} = \max_{y \neq 0} \frac{y^T Dy}{y^T y} = \max_{y \neq 0} \frac{\sum \lambda_i y_i^2}{\sum y_i^2}$$

This expression is a convex combination of the eigenvalues λ_i. The maximum is achieved by placing all weight on the coordinate corresponding to λ_n. The analogous logic applies to λ_1. By restricting $x \perp v_1$, we force $y_1 = 0$, thus the minimum over the remaining subspace is exactly λ_2. □

10.2.2 The Graph Laplacian

For an undirected, unweighted graph $G = (V, E)$ without self-loops or multiple edges, we define the *Laplacian matrix* as $L_G = D_G - A_G$, where A_G is the symmetric adjacency matrix and D_G is the diagonal degree matrix.

Key properties of L_G:

1. L_G is symmetric, as both D_G and A_G are symmetric.
2. $L_G = \sum_{e \in E} L_{\{e\}}$, meaning the Laplacian is a linear sum of Laplacians of individual edges.

To understand the quadratic form $x^T L_G x$, we evaluate the contribution of a single edge $e = \{u, v\}$:

$$x^T L_{\{u,v\}} x = x_u^2 - 2x_u x_v + x_v^2 = (x_u - x_v)^2$$

Summing over all edges yields the fundamental identity:

$$x^T L_G x = \sum_{(u,v) \in E} (x_u - x_v)^2$$

Note that for an indicator vector $\mathbb{1}_S$ of a subset $S \subseteq V$, the quadratic form $\mathbb{1}_S^T L_G \mathbb{1}_S$ exactly counts the number of edges crossing the cut $(S, V \setminus S)$, i.e., $|E(S, S^c)|$.

Since L_G is symmetric and positive semi-definite (as $x^T L_G x \geq 0$), its eigenvalues satisfy $0 \leq \lambda_1 \leq \lambda_2 \leq \cdots \leq \lambda_n$.

Claim 10.2.
- $\lambda_1 = 0$ *is always an eigenvalue, corresponding to the all-ones eigenvector* $\mathbb{1}$.
- $\lambda_2 > 0$ *if and only if the graph* G *is connected.*

Proof. For the vector $\mathbb{1}$ it is easy to see that: $L_G \mathbb{1} = 0$, since the row sums of L_G are zero.

If G is disconnected, let C be a connected component. The indicator vector $\mathbb{1}_C$ satisfies $L_G \mathbb{1}_C = 0$ because no edges leave C. Since $\mathbb{1}_C$ is not a scalar multiple of $\mathbb{1}$ (assuming $C \neq V$), the eigenspace of $\lambda = 0$ has dimension at least 2, implying $\lambda_2 = 0$.

Conversely, assume $\lambda_2 = 0$. There exists an eigenvector $v_2 \perp \mathbb{1}$ such that $v_2^T L_G v_2 = 0$. By the quadratic form identity, $\sum_{(u,v) \in E} ((v_2)_u - (v_2)_v)^2 = 0$. This implies $(v_2)_u = (v_2)_v$ for every edge (u, v). Since G is connected, all entries of v_2 must be identical (because equality holds on every edge), meaning $v_2 \in \text{span}(\mathbb{1})$, which contradicts $v_2 \perp \mathbb{1}$. Thus, $\lambda_2 > 0$. □

Exercise 10.3. *Show that* $\lambda_k = 0$ *if and only if the graph has at least* k *connected components.*

10.3 Normalized Laplacian and Cheeger's Inequality

We have established that $\lambda_2 > 0$ implies connectivity. We now ask whether λ_2 captures the expansion of G. Assuming G is connected, we consider the normalized Laplacian.

Normalizing by degrees gives the random walk matrix $D_G^{-1} L_G = I - D_G^{-1} A_G$, which is not necessarily symmetric. To preserve symmetry, we define the *Normalized Laplacian*:

$$N_G = D_G^{-1/2} L_G D_G^{-1/2} = I - D_G^{-1/2} A_G D_G^{-1/2}$$

N_G shares the same eigenvalues as $D_G^{-1} L_G$. The smallest eigenvalue is $\lambda_1(N_G) = 0$, with corresponding eigenvector $v_1 = \sqrt{\vec{d}}$ (where $\vec{d}$ is the vector of degrees).

Proof.

$$x^T N_G x = \left(D_G^{-1/2}x\right)^T L_G \left(D_G^{-1/2}x\right) = \sum_{(u,v)\in E}\left(\frac{x_u}{\sqrt{\deg(u)}} - \frac{x_v}{\sqrt{\deg(v)}}\right)^2 \geq 0$$

Setting $x = \sqrt{\vec{d}}$ makes every term in the sum zero. □

Claim 10.4.

$$\lambda_2(N_G) = \min_{y\neq 0, y\perp \vec{d}} \frac{y^T L_G y}{y^T D_G y}$$

Proof. We already saw that:

$$\lambda_2(N_G) = \min_{x\neq 0, x\perp \sqrt{\vec{d}}} \frac{x^T N_G x}{x^T x}$$

Apply the substitution $x = D_G^{1/2}y$, which is a bijection since D_G is invertible for graphs with no isolated vertices. The orthogonality constraint $x \perp \sqrt{\vec{d}}$ becomes $(D_G^{1/2}y)^T\sqrt{\vec{d}} = 0 \implies y^T\vec{d} = 0$, meaning $y \perp \vec{d}$. Then we get:

$$\frac{x^T N_G x}{x^T x} = \frac{(D_G^{1/2}y)^T(D_G^{-1/2}L_G D_G^{-1/2})(D_G^{1/2}y)}{(D_G^{1/2}y)^T(D_G^{1/2}y)} = \frac{y^T L_G y}{y^T D_G y}$$

□

This continuous optimization problem is intimately connected to the combinatorial definition of *conductance*:

$$\varphi(G) = \min_{\emptyset\neq S\subset V, \mathrm{vol}(S)\leq \frac{1}{2}\mathrm{vol}(V)} \frac{|E(S,S^c)|}{\mathrm{vol}(S)} = \min_{\emptyset\neq S\subset V, \mathrm{vol}(S)\leq \frac{1}{2}\mathrm{vol}(V)} \frac{\mathbb{1}_S^T L_G \mathbb{1}_S}{\mathbb{1}_S^T D_G \mathbb{1}_S}$$

Theorem 10.5 (Cheeger's Inequality)**.** *For any graph G, the conductance is bounded by the spectral gap:*

$$\frac{1}{2}\lambda_2(N_G) \leq \varphi(G) \leq \sqrt{2\lambda_2(N_G)}$$

This inequality states that computing λ_2 (which can be done efficiently using linear algebra) provides a tight approximation for the NP-hard problem of computing conductance,[1] which allows us to efficiently certify expansion or find a sparse cut.

In the proof we'll use the next exercise:

Exercise 10.6. *Given $\alpha_i, a_i, b_i \geq 0$ (where $\alpha_i, b_i > 0$ for at least one i), we have: $\sum \alpha_i a_i / \sum \alpha_i b_i \geq \min_i a_i/b_i$*

10.3.1 Proof of Cheeger's Inequality

Proof. **Lower Bound ($\lambda_2 \leq 2\varphi$):**
Let S be the subset realizing the conductance, so $\mathrm{vol}(S) \leq \frac{1}{2}\mathrm{vol}(V)$ and $\varphi(G) = \frac{\mathbb{1}_S^T L_G \mathbb{1}_S}{\mathbb{1}_S^T D_G \mathbb{1}_S}$. We

[1] In general the approximation might not be very accurate, but it is sufficient for showing e.g. that the conductance is at least a constant or at least polylogarithmic.

define a vector $x \in \mathbb{R}^V$ by shifting $\mathbb{1}_S$ to make it orthogonal to $\vec{d}$. Let $x = \mathbb{1}_S - \sigma\mathbb{1}$. We require $\vec{d}^T x = 0$:

$$0 = \vec{d}^T(\mathbb{1}_S - \sigma\mathbb{1}) = \operatorname{vol}(S) - \sigma\operatorname{vol}(V) \implies \sigma = \frac{\operatorname{vol}(S)}{\operatorname{vol}(V)}$$

We now evaluate $\frac{x^T L_G x}{x^T D_G x}$. Since $\mathbb{1} \in \ker(L_G)$, the numerator is unchanged: $x^T L_G x = \mathbb{1}_S^T L_G \mathbb{1}_S$. As for the denominator:

$$\begin{aligned} x^T D_G x &= (\mathbb{1}_S - \sigma\mathbb{1})^T D_G(\mathbb{1}_S - \sigma\mathbb{1}) \\ &= \mathbb{1}_S^T D_G \mathbb{1}_S - 2\sigma\mathbb{1}_S^T D_G \mathbb{1} + \sigma^2 \mathbb{1}^T D_G \mathbb{1} \\ &= \operatorname{vol}(S) - 2\sigma\operatorname{vol}(S) + \sigma^2\operatorname{vol}(V) \end{aligned}$$

Recall that $\sigma = \frac{\operatorname{vol}(S)}{\operatorname{vol}(V)} \iff \operatorname{vol}(V) = \frac{\operatorname{vol}(S)}{\sigma}$, so we get

$$\begin{aligned} &= \operatorname{vol}(S) - \sigma\operatorname{vol}(S) \\ &= (1-\sigma)\operatorname{vol}(S) \end{aligned}$$

Since $\operatorname{vol}(S) \le \frac{1}{2}\operatorname{vol}(V)$, we have $\sigma \le 1/2$, so $(1-\sigma) \ge 1/2$. Thus:

$$\lambda_2(N_G) \le \frac{x^T L_G x}{x^T D_G x} = \frac{\mathbb{1}_S^T L_G \mathbb{1}_S}{(1-\sigma)\operatorname{vol}(S)} = \frac{\varphi(G)}{1-\sigma} \le 2\varphi(G)$$

Upper Bound ($\varphi \le \sqrt{2\lambda_2}$):
We constructively extract a combinatorial cut from a general eigenvector corresponding to the eigenvalue λ_2.
Let $x \perp \vec{d}$ be the vector achieving $\lambda_2(N_G) = \frac{x^T L_G x}{x^T D_G x}$. Without loss of generality, sort the vertices such that $x_1 \le x_2 \le \cdots \le x_n$. Let j be the minimal index such that $\sum_{i=1}^{j} \deg(v_i) > \frac{1}{2}\operatorname{vol}(V)$. We center the vector around x_j by defining $y = x - x_j\mathbb{1}$. The numerator remains identical: $y^T L_G y = x^T L_G x$ (once again since $\mathbb{1} \in \ker(L_G)$). The denominator expands to:

$$\begin{aligned} y^T D_G y &= (x - x_j\mathbb{1})^T D_G(x - x_j\mathbb{1}) \\ &= x^T D_G x - 2x_j(x^T\vec{d}) + x_j^2\operatorname{vol}(V) \\ &= x^T D_G x - 2x_j \cdot 0 + x_j^2\operatorname{vol}(V) \\ &= x^T D_G x + x_j^2\operatorname{vol}(V) \\ &\ge x^T D_G x \end{aligned}$$

Thus, centering improves (or maintains) the quotient: $\frac{y^T L_G y}{y^T D_G y} \le \lambda_2(N_G)$.

We split y into positive and negative parts: $y = y^+ - y^-$, where $y_i^+ = \max(y_i, 0)$ and $y_i^- = \max(-y_i, 0)$. Notice $y_i^+ y_i^- = 0$ for all i. For the denominator:

$$y^T D_G y = (y^+)^T D_G y^+ + (y^-)^T D_G y^-$$

For the numerator:

$$\begin{aligned} y^T L_G y &= \sum_{(u,v)\in E} ((y_u^+ - y_v^+) - (y_u^- - y_v^-))^2 \\ &= (y^+)^T L_G y^+ + (y^-)^T L_G y^- - 2\sum_{(u,v)\in E} (y_u^+ - y_v^+)(y_u^- - y_v^-) \end{aligned}$$

Observe that $(y_u^+ - y_v^+)(y_u^- - y_v^-) \le 0$ for all edges. (If y_u, y_v share the same sign, one factor is zero. If they have opposite signs, the product is strictly negative). Thus, dropping the cross-term yields:

$$y^T L_G y \ge (y^+)^T L_G y^+ + (y^-)^T L_G y^-$$

By the exercise we saw, we know that $\frac{A+B}{C+D} \ge \min\left(\frac{A}{C}, \frac{B}{D}\right)$, so one of the vectors y^+ or y^- must achieve a quotient $\le \lambda_2$. WLOG, assume it is y^+. Note that the support of y^+ has volume $\le \frac{1}{2}\operatorname{vol}(V)$ by our choice of the index j.

We normalize y^+ so its maximum value is 1. We apply a randomized rounding technique: pick a threshold τ uniformly at random in $[0,1]$, and define the cut $S_\tau = \{v_i \mid (y_i^+)^2 \ge \tau\}$. We compute the expectations over τ. For the denominator:

$$\mathbb{E}_\tau[\mathbb{1}_{S_\tau}^T D_G \mathbb{1}_{S_\tau}] = \sum_{i\in V} \deg(v_i) \Pr_\tau[(y_i^+)^2 \ge \tau] = \sum_{i\in V} \deg(v_i)(y_i^+)^2 = (y^+)^T D_G y^+$$

For the numerator:

$$\begin{aligned} \mathbb{E}_\tau[\mathbb{1}_{S_\tau}^T L_G \mathbb{1}_{S_\tau}] &= \sum_{(u,v)\in E} \mathbb{E}_\tau\left[([\mathbb{1}_{S_\tau}]_u - [\mathbb{1}_{S_\tau}]_v)^2\right] \\ &= \sum_{(u,v)\in E} \Pr_\tau[\text{exactly one of } u, v \in S_\tau] \\ &= \sum_{(u,v)\in E} \left|(y_u^+)^2 - (y_v^+)^2\right| \end{aligned}$$

Factoring the difference of squares and applying the Cauchy-Schwarz inequality:

$$\begin{aligned} \mathbb{E}_\tau[\mathbb{1}_{S_\tau}^T L_G \mathbb{1}_{S_\tau}] &= \sum_{(u,v)\in E} |y_u^+ - y_v^+|(y_u^+ + y_v^+) \\ &\le \sqrt{\sum_{(u,v)\in E} (y_u^+ - y_v^+)^2} \sqrt{\sum_{(u,v)\in E} (y_u^+ + y_v^+)^2} \end{aligned}$$

The first factor is $\sqrt{(y^+)^T L_G y^+}$. For the second factor, we use the bound $(a+b)^2 \le 2(a^2+b^2)$:

$$\sum_{(u,v)\in E} (y_u^+ + y_v^+)^2 \le 2\sum_{(u,v)\in E} ((y_u^+)^2 + (y_v^+)^2) = 2\sum_{i\in V} \deg(v_i)(y_i^+)^2 = 2(y^+)^T D_G y^+$$

Thus:

$$\mathbb{E}_\tau[\mathbb{1}_S^T L_G \mathbb{1}_S] \le \sqrt{(y^+)^T L_G y^+} \cdot \sqrt{2(y^+)^T D_G y^+}$$

Summing it all up, we get:

$$\frac{\mathbb{E}_\tau[\mathbb{1}_S^T L_G \mathbb{1}_S]}{\mathbb{E}_\tau[\mathbb{1}_S^T D_G \mathbb{1}_S]} \le \frac{\sqrt{2}\sqrt{(y^+)^T L_G y^+ (y^+)^T D_G y^+}}{(y^+)^T D_G y^+} = \sqrt{2\frac{(y^+)^T L_G y^+}{(y^+)^T D_G y^+}} \le \sqrt{2\lambda_2}$$

Hence, there exists a threshold τ whose conductance is at most the above ratio, defining a cut S such that $\varphi(S) \leq \sqrt{2\lambda_2}$. Algorithmically, since the possible cuts are determined by the sorted order of the coordinates of y^+ (and try also the options of y^-), we only need to test the n possible suffix sets. This combinatorial step takes $O(m + n \log n)$ time. □

Observation 10.7 (Algorithmic Computation of Eigenvectors)**.** *Computing λ_2 and its eigenvector via full eigendecomposition takes $O(n^3)$ time, which is too slow for our $O(m)$ goal. However, we can approximate the dominant eigenvector iteratively using the Power Method. For a symmetric matrix A, iteratively applying A to a random vector v ($A^k v$) amplifies the component of the largest eigenvalue. To isolate λ_{n-1}, we simply maintain orthogonality to the known eigenvector v_n (or $\mathbb{1}$). Since A is the adjacency structure, computing Av is equivalent to multiply over all edges, requiring $O(m)$ time per iteration. We can perform it to get λ_2, v_2 instead of λ_{n-1}, v_{n-1}. This reduces the eigenvector approximation complexity to $\tilde{O}(m)$.*

Exercise 10.8. *Complete the details.*

Bibliography

[1] Amir Abboud, Karl Bringmann, Seri Khoury, and Or Zamir. Hardness of approximation in p via short cycle removal: cycle detection, distance oracles, and beyond. In *Proceedings of the 54th Annual ACM SIGACT Symposium on Theory of Computing*, STOC 2022, page 1487–1500, New York, NY, USA, 2022. Association for Computing Machinery.

[2] Noga Alon, Raphael Yuster, and Uri Zwick. Color-coding. *Journal of the ACM (JACM)*, 42(4):844–856, 1995.

[3] Andreas Björklund, Thore Husfeldt, and Mikko Koivisto. Set partitioning via inclusion-exclusion. *SIAM Journal on Computing*, 39(2):546–563, 2009. DOI: 10.1137/070683933.

[4] John A Bondy and Miklós Simonovits. Cycles of even length in graphs. *Journal of Combinatorial Theory, Series B*, 16(2):97–105, 1974.

[5] Otakar Borvka. O jistém problému minimálním. 1926.

[6] Bernard Chazelle. Chazelle, b.: A minimum spanning tree algorithm with inverse-ackermann type complexity. journal of the acm 47(6), 1028-1047. *J. ACM*, 47:1028–1047, 11 2000.

[7] Pál Erdős and Arthur H. Stone. On the structure of linear graphs. *Bulletin of the American Mathematical Society*, 52(12):1087–1091, 1946.

[8] M. L. Fredman and R. E. Tarjan. Fibonacci heaps and their uses in improved network optimization algorithms. In *Proceedings of the 25th Annual Symposium OnFoundations of Computer Science, 1984*, SFCS '84, page 338–346, USA, 1984. IEEE Computer Society.

[9] H.N. Gabow. A matroid approach to finding edge connectivity and packing arborescences. *Journal of Computer and System Sciences*, 50(2):259–273, 1995.

[10] Oded Goldreich, Shari Goldwasser, and Dana Ron. Property testing and its connection to learning and approximation. *Journal of the ACM (JACM)*, 45(4):653–750, 1998.

[11] Thomas Dueholm Hansen, Haim Kaplan, Or Zamir, and Uri Zwick. Faster k-sat algorithms using biased-ppsz. In *Proceedings of the 51st Annual ACM SIGACT Symposium on Theory of Computing*, pages 578–589, 2019.

[12] Russell Impagliazzo and Ramamohan Paturi. On the complexity of k-sat. *Journal of Computer and System Sciences*, 62(2):367–375, 2001.

[13] David R. Karger, Philip N. Klein, and Robert E. Tarjan. A randomized linear-time algorithm to find minimum spanning trees. *J. ACM*, 42(2):321–328, March 1995.

[14] Joseph B. Kruskal. On the shortest spanning subtree of a graph and the traveling salesman problem. *Proceedings of the American Mathematical Society*, 7(1):48–50, 1956.

[15] Willem Mantel. Problem 28. *Wiskundige Opgaven*, 10:60–61, 1907.

[16] Burkhard Monien and Ewald Speckenmeyer. Solving satisfiability in less than 2n steps. *Discrete Applied Mathematics*, 10(3):287–295, 1985.

[17] R. C. Prim. Shortest connection networks and some generalizations. *Bell System Technical Journal*, 36(6):1389–1401, 1957.

[18] Neil Robertson and Paul Seymour. Graph minors. xx. wagner's conjecture. *J. Comb. Theory, Ser. B*, 92:325–357, 11 2004.

[19] Gian-Carlo Rota. On the foundations of combinatorial theory. i. theory of möbius functions. *Zeitschrift für Wahrscheinlichkeitstheorie und Verwandte Gebiete*, 2(4):340–368, 1964.

[20] Endre Szemerédi. Regular partitions of graphs. In *Problèmes combinatoires et théorie des graphes*, volume 260, pages 399–401, 1978.

[21] R. Michael Tanner. A recursive approach to low complexity codes. *IEEE Transactions on Information Theory*, 27(5):533–547, 1981.

[22] Pál Turán. Über ein extremalproblem in der graphentheorie. *Matematikai ´es Fizikai Lapok*, 48:436–452, 1941.

[23] Virginia Vassilevska Williams and Yinzhan Xu. Monochromatic Triangles, Triangle Listing and APSP . In *2020 IEEE 61st Annual Symposium on Foundations of Computer Science (FOCS)*, pages 786–797, Los Alamitos, CA, USA, November 2020. IEEE Computer Society.

[24] Frank Yates. The design and analysis of factorial experiments. Technical Communication 35, Imperial Bureau of Soil Science, Harpenden, 1937.

[25] Raphael Yuster and Uri Zwick. Finding even cycles even faster. In *International Colloquium on Automata, Languages, and Programming*, pages 532–543. Springer, 1994.